

AI-Powered Digital Image Forensics Platform

A Project Report On

"METALENS"

Digital Image Forensics & Evidence Verification

Submitted in partial fulfillment of the requirements for the certification of

Full Stack Web Development (MERN)

[Anudeep Foundation]

Submitted By:

Rutuja Shivaji Shingote

AF CODE:- AF05264851

Under the guidance of

Prof. [MRS.Rajshree Thete]

Academic Year 2026

Index

Index Sr. no	Topic	Page no
1	Title of Project	1
2	Acknowledgement	3
3	Abstract	4
4	Introduction	5
5	Objective	6
6	System Analysis	7
7	System Design	25
8	Screenshots	31
9	Coding	42
10	Testing	52
11	Report	55
12	Future Scope	57
13	Conclusion	58
14	Bibliography	59
15	References	60

Acknowledgement

Name	Enrollment No
Rutuja Shivaji Shingote	AF05264851

Under the Guidance.

We are thankful to my project guide for guiding me to complete the Project.

His suggestions and valuable information regarding the formation of the Project Report have provided me a lot of help in completing the Project and its related topics.

We are also thankful to my family member and friends who were always there to provide support and moral boost up.

Abstract

The project called MetaLens is about creating a web platform to analyze digital images and uncover hidden information that is not visible to the naked eye. This system replaces the traditional manual method of checking image authenticity with an automated forensic analysis pipeline that extracts metadata, verifies image integrity, and generates evidence reports.

This platform lets users upload any image and instantly receive a comprehensive forensic report containing EXIF metadata, GPS location, camera details, timestamps, tampering indicators, and a cryptographic SHA-256 fingerprint. It also uses an AI-powered vision model to describe visible content in the image, helping users understand what is present without manual inspection. Every analysis is saved to a secure database, allowing users to revisit past reports anytime through their personal history dashboard.

The system is simple, easy to use, and works smoothly on both desktop and mobile devices. It provides secure user accounts, real-time metadata extraction, automatic chain-of-custody generation, and downloadable PDF reports. MetaLens helps journalists, investigators, insurance agents, and students verify image authenticity in seconds instead of hours. The platform focuses on accuracy, transparency, and making digital evidence trustworthy for real-world use.

1. Introduction

In today's digital era, we are surrounded by images at every moment. From smartphone photography to social media posts, CCTV footage to news reports, digital images have become one of the most common and influential forms of communication. Every photo we capture or share carries invisible information embedded within it — camera details, GPS coordinates, timestamps, and software signatures — known as metadata. While most people never notice it, this hidden data can reveal critical facts about an image's origin, authenticity, and history. As technology advances, so does the ease of manipulating images, making it increasingly difficult to trust what we see. Verifying the authenticity of digital images has therefore become a serious challenge for journalists, law enforcement, insurance companies, and legal professionals.

Our project aims to develop MetaLens, a web-based Digital Image Forensics Platform that makes it easy for anyone to uncover the truth hidden inside an image. The platform allows users to upload any digital photo and instantly receive a comprehensive forensic report containing EXIF metadata, GPS location, camera information, timestamps, tampering indicators, and a cryptographic SHA-256 fingerprint. An AI-powered vision model analyzes the visible content of the image, while advanced computer vision techniques such as FFT frequency analysis, Error Level Analysis, and noise variance detection identify signs of manipulation. Every analysis is securely stored in a database, allowing users to revisit their past reports through a personal history dashboard. By combining metadata extraction, computer vision, artificial intelligence, and cryptographic verification into a single platform, MetaLens provides a fast, reliable, and accessible solution for digital image authenticity — an essential need in today's world of visual information.

1.1 Objectives of the Present Work

The objectives of this project are as follows:

- o To develop a web application for Digital Image Forensics that can help users verify the authenticity of any image.
- o To extract hidden EXIF metadata such as camera model, lens details, ISO, shutter speed, and software information from images.
- o To recover embedded GPS coordinates and convert them into a human-readable address using reverse geocoding.
- o To detect possible tampering and manipulation through advanced techniques like Error Level Analysis (ELA), FFT frequency analysis, and noise inconsistency detection.
- o To generate a cryptographic SHA-256 fingerprint of every image for secure evidence verification and tamper-proof record keeping.
- o To use a locally-hosted AI vision model for automatic description of visible content in an image without relying on paid cloud APIs.
- o To build a complete chain-of-custody timeline showing how an image traveled from capture to analysis.
- o To generate downloadable PDF forensic reports suitable for professional and legal documentation.
- o To store every analysis securely in a database so users can revisit their report history anytime.
- o To provide secure user authentication so that each user's evidence remains private and personal.
- o To make image forensic analysis fast, accessible, and affordable for journalists, investigators, insurance agents, students, and anyone who needs to verify digital evidence.
- o To deliver a clean, modern, and responsive interface that works smoothly on both desktop and mobile devices.

System Analysis

3.1 PROBLEM DEFINITION

Many digital images today cannot be trusted at first glance. Photos are easily edited using freely available tools, and most people have no way of knowing whether an image is original or manipulated. Traditional methods of image verification require technical expertise and expensive software, making them inaccessible to journalists, insurance agents, students, and small businesses. Additionally, when images are shared on messaging apps and social media, their metadata is often stripped automatically, leaving no trace of the original source, location, or capture time.

This project solves these problems by providing an easy-to-use web platform where anyone can upload an image and receive a detailed forensic report within seconds. The system extracts hidden metadata, recovers GPS coordinates, detects signs of manipulation, generates a cryptographic fingerprint, and uses AI to describe the visible content. It makes professional-grade image forensics accessible to non-experts without requiring any software installation.

3.2 Preliminary Investigation

PURPOSE:-

MetaLens is designed to make digital image verification fast, reliable, and accessible through a web-based platform. It aims to replace manual and expensive forensic tools by allowing users to upload any image and instantly receive a comprehensive analysis covering metadata, GPS, tampering indicators, and cryptographic integrity. The platform also lets users keep a personal history of their analyses for future reference.

BENEFITS:-

The platform provides several advantages:

- **Instant Image Analysis:** Users can upload an image and receive a full forensic report in seconds, without any technical knowledge.
- **Comprehensive Metadata Extraction:** Camera details, timestamps, GPS coordinates, and software signatures are extracted automatically.
- **Tamper Detection:** Advanced techniques like ELA, FFT, and noise variance reveal signs of manipulation.
- **Cryptographic Verification:** Every image gets a SHA-256 fingerprint for secure evidence tracking.

- **AI-Powered Visual Insight:** A local AI model describes visible content, objects, and text within the image.
- **Personal Report History:** Users can revisit their past analyses anytime through a secure dashboard.
- **Downloadable PDF Reports:** Professional forensic reports suitable for documentation and legal use.

PROPOSED SYSTEM :-

The proposed system offers a structured, reliable, and automated approach to digital image forensics:

- Users can upload images, receive instant forensic analysis, and download PDF reports.
- The system extracts EXIF metadata, GPS location, and timestamps automatically without user intervention.
- Tampering is detected through multiple computer vision techniques (ELA, FFT, noise inconsistency, compression artifact analysis).
- An AI vision model provides descriptive analysis of the visible content inside the image.
- Every report is securely stored in a database and linked to the user's account for future access.
- The platform is designed to be simple, responsive, and efficient, ensuring a smooth experience for both technical and non-technical users.

3.3 Feasibility Study

The feasibility study evaluates whether the MetaLens Digital Image Forensics Platform is practical, achievable, and beneficial. This assessment ensures the system can be successfully implemented within available resources, technology, and constraints.

TYPES OF FEASIBILITY ANALYSIS:-

• Technical Feasibility

- ✓ The platform is built using Python (Flask) for the backend and React (Vite) for the frontend, making it compatible with modern web technologies.
- ✓ The database is managed using MongoDB, ensuring flexible and scalable storage for forensic reports.
- ✓ Image processing is handled using Pillow and NumPy, while metadata extraction uses ExifTool — a widely trusted forensic tool.

✓ AI-powered visual analysis is integrated using Ollama running the Moondream vision model locally, avoiding any paid cloud APIs.

✓ The system is designed to run on standard hardware without requiring specialized forensic equipment.

• Economic Feasibility

✓ All technologies used are open-source — Flask, React, MongoDB, Ollama, and ExifTool — resulting in zero software licensing costs.

✓ The AI model runs locally, eliminating recurring cloud API expenses that would otherwise be required for image analysis.

✓ The platform saves organizations significant time and money by replacing manual verification with automated analysis.

• Operational Feasibility

✓ Users can access the platform through any modern web browser on desktop, tablet, or mobile.

✓ The interface is intuitive — users simply upload an image and receive a full report without any technical training.

✓ Reports are organized in a searchable history, making it easy to find and revisit past analyses.

✓ Secure authentication ensures each user's evidence remains private and protected.

• Schedule Feasibility

✓ The development timeline is realistic, covering design, coding, testing, and deployment within the project duration.

✓ Modular development ensures that each component — metadata extraction, AI analysis, authentication, and reporting — was built and tested incrementally.

✓ The final system was successfully delivered within the planned timeframe.

• Social Feasibility

✓ The project promotes trust in digital media by giving everyone access to professional-grade forensic tools.

✓ It helps journalists verify images before publishing, prevents the spread of manipulated photos, and supports legal and insurance professionals in validating evidence.

✓ The platform is accessible to all users, regardless of their technical background or expertise.

3.4 Project Planning

PURPOSE OF PROJECT PLANNING

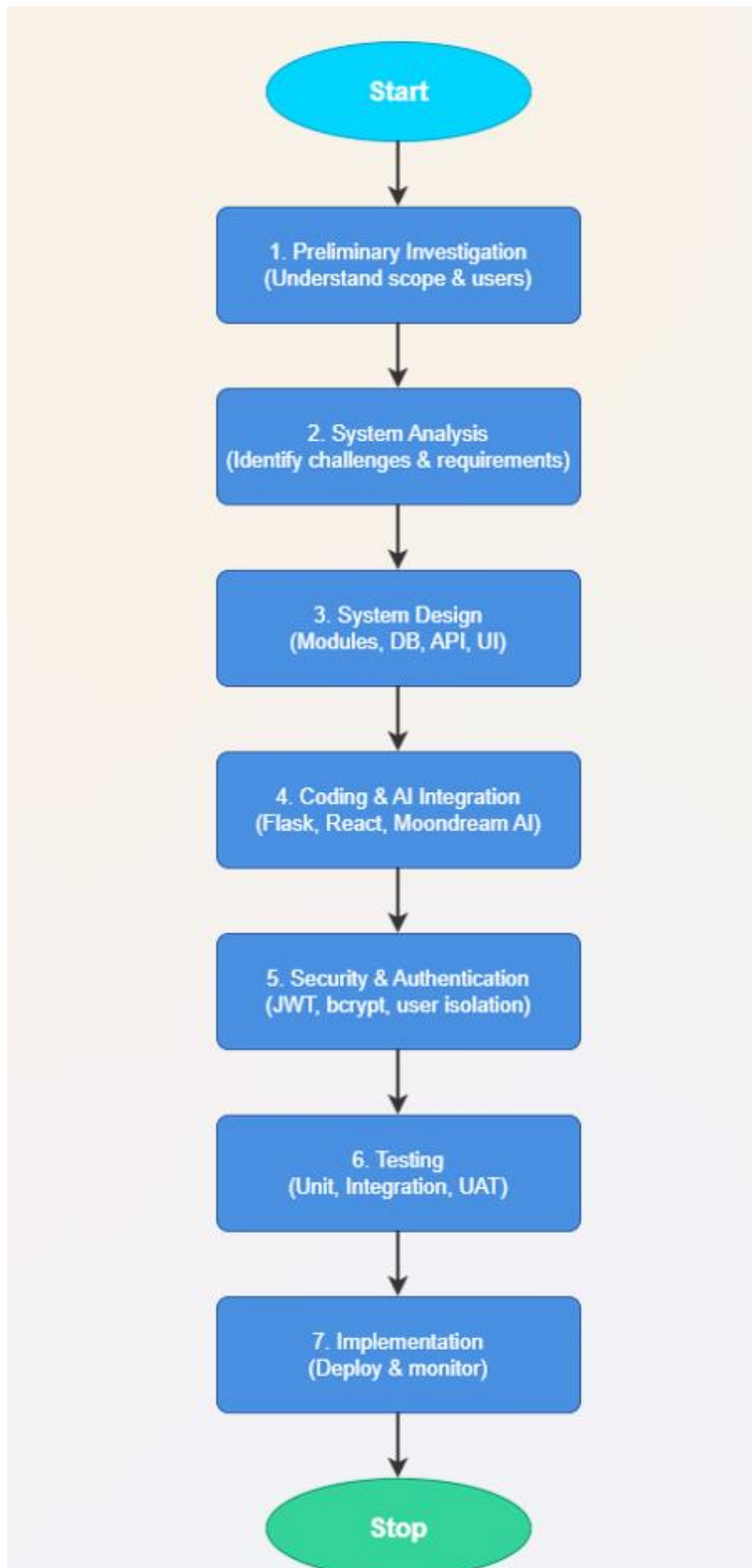
Project planning ensures that the development of the MetaLens Digital Image Forensics Platform follows a structured approach. It helps define the workflow, resource allocation, timeline, and risks involved to deliver the system efficiently.

PHASES COVERED IN THE PLAN

The planning is divided into different phases to ensure smooth execution:

1. Preliminary Investigation – Understanding the project scope, target users (journalists, investigators, students), and forensic objectives.
2. System Analysis – Identifying challenges, gathering requirements, and defining the analysis pipeline (metadata, GPS, tampering, AI).
3. System Design – Structuring modules, database schema, API routes, and UI/UX design for the platform.
4. Coding & AI Integration – Developing the platform using Flask, React, MongoDB, and integrating the Moondream AI model via Ollama.
5. Security & Authentication – Implementing JWT-based authentication, bcrypt password hashing, and user-isolated report storage.
6. Testing – Performing unit testing, integration testing, and user acceptance testing on all forensic techniques.
7. Implementation – Deploying the final system and ensuring smooth operation across desktop and mobile devices.

DIAGRAM:-



3.5 Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) outlines the fundamental requirements of the MetaLens Digital Image Forensics Platform to ensure efficient functionality, usability, and maintainability.

SYSTEM OVERVIEW

MetaLens allows users to upload digital images and receive comprehensive forensic analysis covering metadata, GPS, tampering indicators, AI-driven content description, and cryptographic integrity. The system is structured into three modules:

1. **User Module** – Enables users to sign up, log in, upload images, view forensic reports, and access personal analysis history.
2. **Analysis Module** – Performs automated forensic extraction using ExifTool, Pillow, and NumPy, running seven forensic techniques simultaneously.
3. **AI & Reporting Module** – Uses a locally-hosted vision model (Moondream via Ollama) to describe visible image content and generates downloadable PDF forensic reports.

SOFTWARE & HARDWARE REQUIREMENTS

Software Requirements

- **Frontend:** React 18, Vite, React Router v6, Axios for the responsive UI
- **Backend:** Python (Flask) for routing, image processing, and API handling
- **Database:** MongoDB for storing user accounts and forensic reports
- **Authentication:** JWT (PyJWT) for session tokens, bcrypt for password hashing
- **Image Processing:** Pillow, NumPy for ELA, FFT, and noise analysis
- **Metadata Extraction:** ExifTool for EXIF, GPS, and embedded metadata
- **AI Model:** Ollama running Moondream (1.8B vision-language model) locally
- **Geocoding:** Nominatim (OpenStreetMap) API for reverse geocoding
- **Configuration:** python-dotenv for managing environment secrets

Hardware Requirements

- **Processor:** Intel i5 / Ryzen 5 or higher (for AI model inference)
- **RAM:** Minimum 8GB (16GB recommended for smooth AI analysis)
- **Storage:** At least 10GB for the database, AI models, and temporary files
- **Connectivity:** Internet access for reverse geocoding and package setup

3.6 Functional Requirements

1. User Module

Users can:

- Create a secure account using name, email, and password (minimum 8 characters).
- Log in and receive a JWT token valid for 7 days.
- Upload images via drag-and-drop or file browser (JPG, PNG, WEBP, up to 50MB).
- View comprehensive forensic reports covering EXIF, GPS, timestamp, platform, pixel, AI, and verdict sections.
- Download forensic reports as PDF for documentation or legal use.
- Generate and copy SHA-256 fingerprints for evidence tracking.
- Share reports via Email, WhatsApp, or native OS share.
- Access a personal history of all past analyses.
- Search and filter historical reports by filename or verdict.
- Delete old reports from their history.

2. Analysis Module

The analysis module automatically performs the following:

- **EXIF Extraction** – Camera make, model, lens, ISO, aperture, shutter speed, and software version.
- **GPS Recovery** – Latitude, longitude, altitude, and reverse-geocoded address.
- **Timestamp Analysis** – Original capture time, file modification time, and reconstructed date from filename.
- **Platform Detection** – Identifies whether the image came from a camera, WhatsApp, Instagram, Facebook, or a screenshot.
- **Image Integrity Analysis** – Runs Error Level Analysis (ELA), FFT frequency analysis, noise inconsistency detection, and compression artifact analysis.
- **Pixel Analysis** – Dimensions, aspect ratio, entropy, and megapixel count.
- **Cryptographic Hashing** – Generates SHA-256 fingerprints of the image file.

3. AI & Reporting Module

The AI and reporting module provides:

- **AI Visual Analysis** – Locally-hosted Moondream vision model describes visible content, objects, scenes, and text in the image.
- **Verdict Generation** – Rule-based expert system calculates a confidence score (0–100) and classifies the image as AUTHENTIC, PARTIAL, or TAMPERED.
- **Chain-of-Custody Timeline** – Automatically generates a 4-step timeline from capture to analysis.
- **PDF Report Generation** – Produces a downloadable forensic report with all findings, hash, and AI analysis.
- **Report Sharing** – Enables sharing via Email, WhatsApp, native share, copy-to-clipboard, and JSON export.

3.7 Software Engineering Paradigm

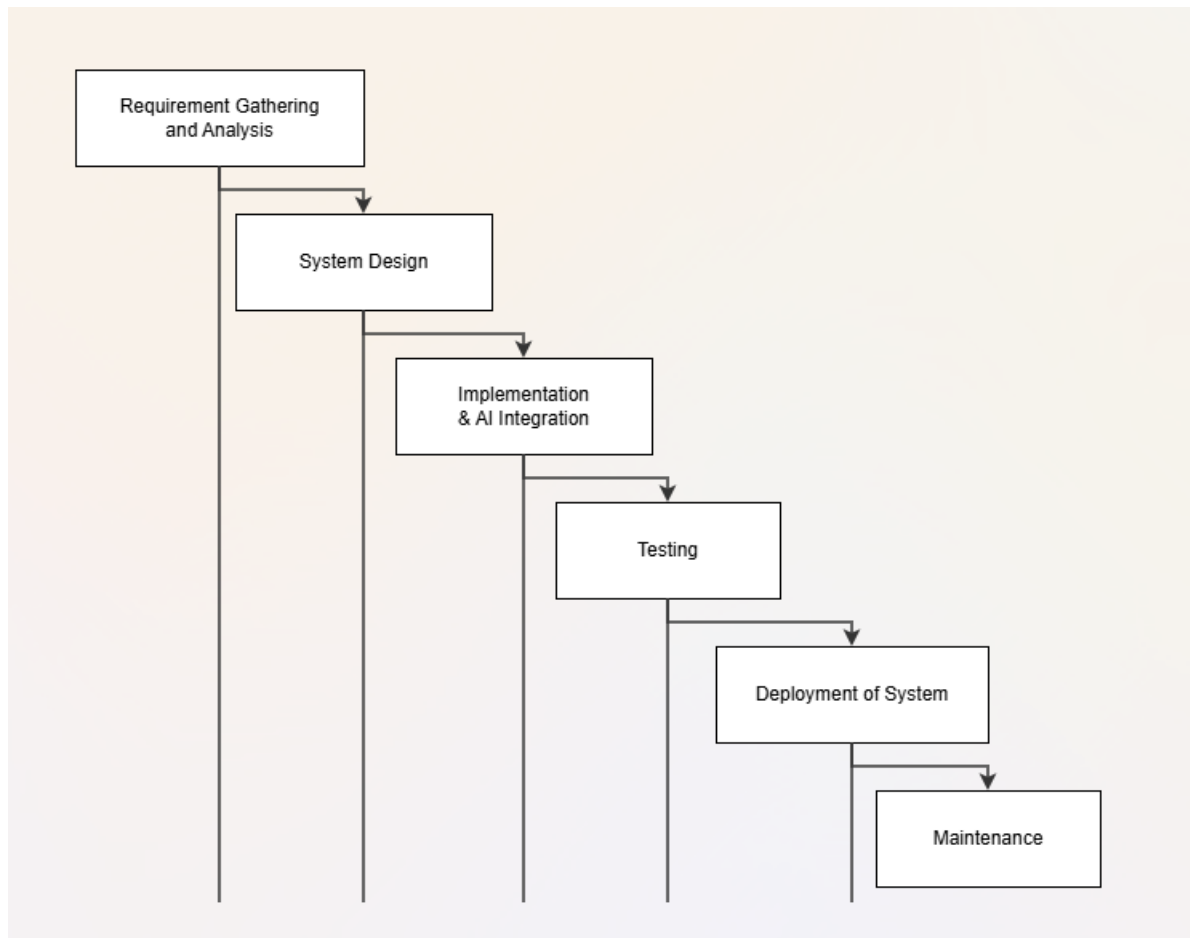
The development of the MetaLens Digital Image Forensics Platform follows a structured approach to ensure efficiency, reliability, and maintainability. The chosen paradigm helps streamline the project by defining clear phases while allowing iterative improvements.

Development Model: Adapted Waterfall Model

The Waterfall Model is traditionally a linear approach, but for this project, an iterative feedback mechanism is incorporated. This helps refine earlier phases based on insights gathered during implementation.

Key Adaptations in the Waterfall Model:

1. **Structured Phase Progression** – Each stage follows a defined sequence, ensuring clarity in execution from requirement gathering to final deployment.
2. **Iterative Refinements** – Feedback loops allow adjustments, especially between coding and testing, as new forensic techniques were added or refined.
3. **Defined Milestones** – Each stage reaches completion before moving to the next phase, ensuring that backend APIs, authentication, and the frontend were fully functional before integration.
4. **Flexible Adjustments** – Overlapping is permitted when necessary to enhance efficiency, particularly during AI model integration and UI polishing.
5. **Component-Based Modularity** – Each forensic technique (EXIF, GPS, ELA, FFT, noise, AI) was developed as an independent module, making it easy to test, debug, and extend.



Phases of Development

1. Requirement Analysis & System Study

- o Identifying project goals, forensic challenges, and functional specifications.
- o Gathering user requirements (journalists, investigators, students) and defining core forensic techniques.

2. System Design

- o Structuring the MongoDB database, REST API modules, and overall architecture.
- o Designing a dark-themed, sci-fi-inspired user interface with responsive layouts for desktop and mobile.

3. Implementation (Coding)

- o Backend development using Python (Flask) for image processing, metadata extraction, and AI integration.

- o Frontend design using React (Vite) with modern hooks and routing.
- o Database integration with MongoDB for user accounts and reports.
- o AI vision analysis integrated using Ollama running the Moondream model locally.
- o Image processing using Pillow, NumPy, and ExifTool.

4. Testing & Debugging

- o Unit testing, integration testing, and usability checks for all forensic techniques.
- o Debugging for performance improvements, especially around AI inference and image handling.
- o Security testing for authentication (JWT), password hashing (bcrypt), and route protection.

5. Deployment & Maintenance

- o Hosting on a scalable environment with backend and frontend separated.
- o Continuous updates for new forensic techniques, improved AI accuracy, and additional export formats.

3.8 Data Flow Diagram

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It can be manual, automated, or a combination of both.

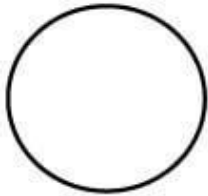
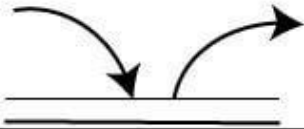
It shows how data enters and leaves the system, what changes the information, and where data is stored.

The objective of a DFD is to show the scope and boundaries of a system as a whole. It may be used as a communication tool between a system analyst and any person who plays a part in the order that acts as a starting point for redesigning a system. The DFD is also called as a data flow graph or bubble chart.

The following observations about DFDs are essential:

1. All names should be unique. This makes it easier to refer to elements in the DFD.
2. Remember that DFD is not a flow chart. A flow chart represents the order of events; arrows in DFD represent flowing data. A DFD does not imply any order of events.
3. Suppress logical decisions. If we ever have the urge to draw a diamond-shaped box in a DFD, suppress that urge. A diamond-shaped box is used in flow charts to represent decision points with multiple exits, which is not a DFD.
4. Do not become bogged down with details. Defer error conditions and error handling until the end of the analysis.

Standard symbols for DFDs are derived from the electric circuit diagram analysis and are shown in fig.

Symbol	Name	Function
	Data flow	Used to Connect Processes to each , other , to sources or Sinks; te arrow head indicates direction of data flow.
	Process	Performs Some transformation of Input data to yield output data.
	Source of Sink (External Entity)	A Source of System inputs or Sink of System outputs.
	Data Store	A repository of data; the arrow heads indicate net inputs and net outputs to store.

Symbols for Data Flow Diagrams

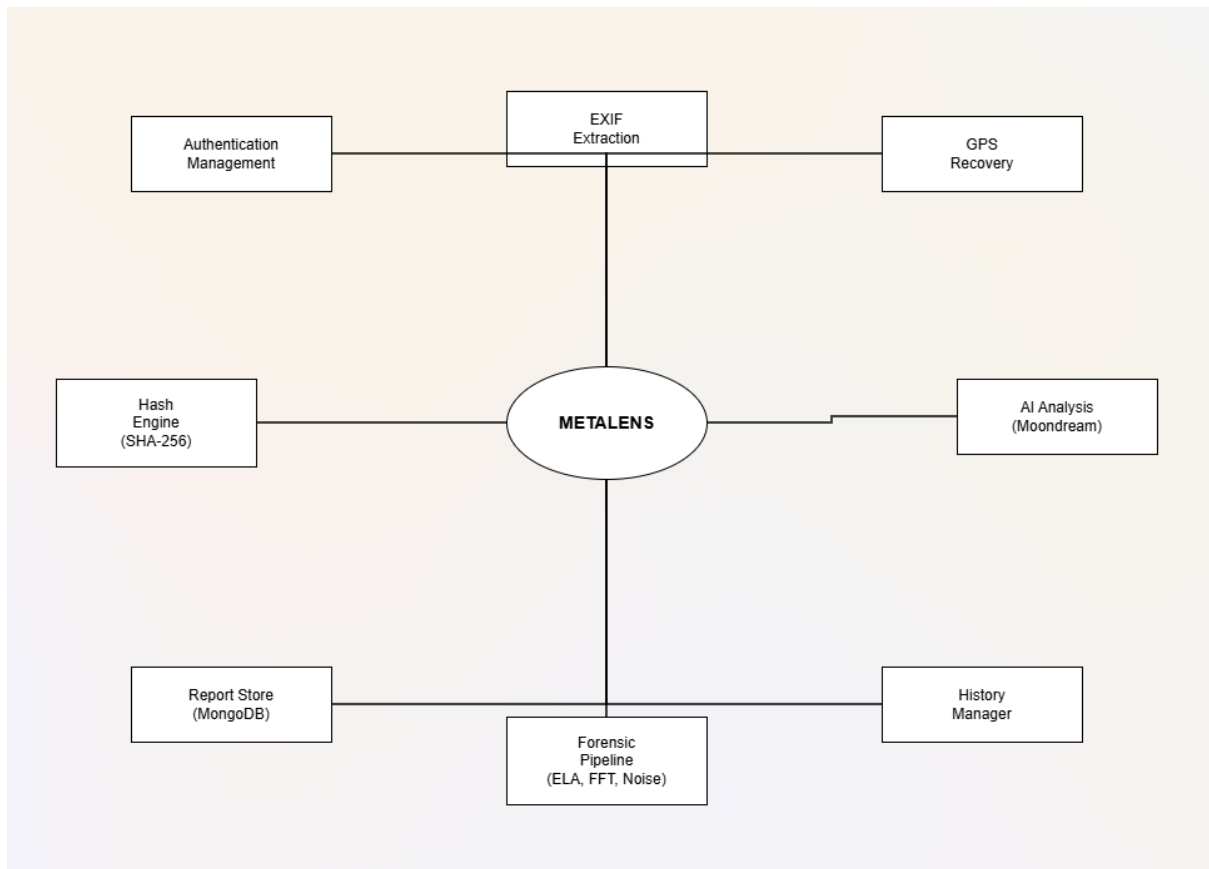
Circle: A circle (bubble) shows a process that transforms data inputs into data outputs.

Data Flow: A curved line shows the flow of data into or out of a process or data store.

Data Store: A set of parallel lines shows a place for the collection of data items. A data store indicates that the data is stored which can be used at a later stage or by the other processes in a different order. The data store can have an element or group of elements.

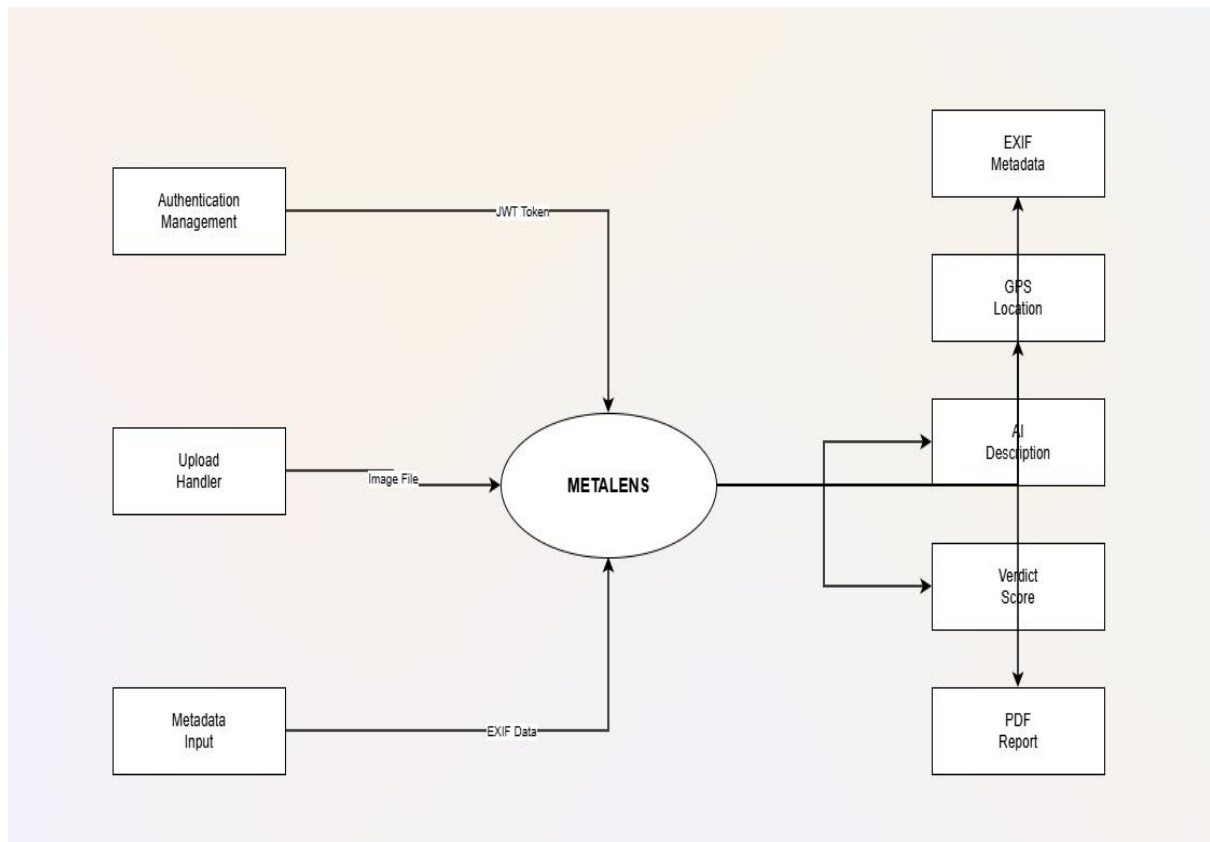
Source or Sink: Source or Sink is an external entity and acts as a source of system inputs or sink of system outputs.

ZERO LEVEL DFD:-



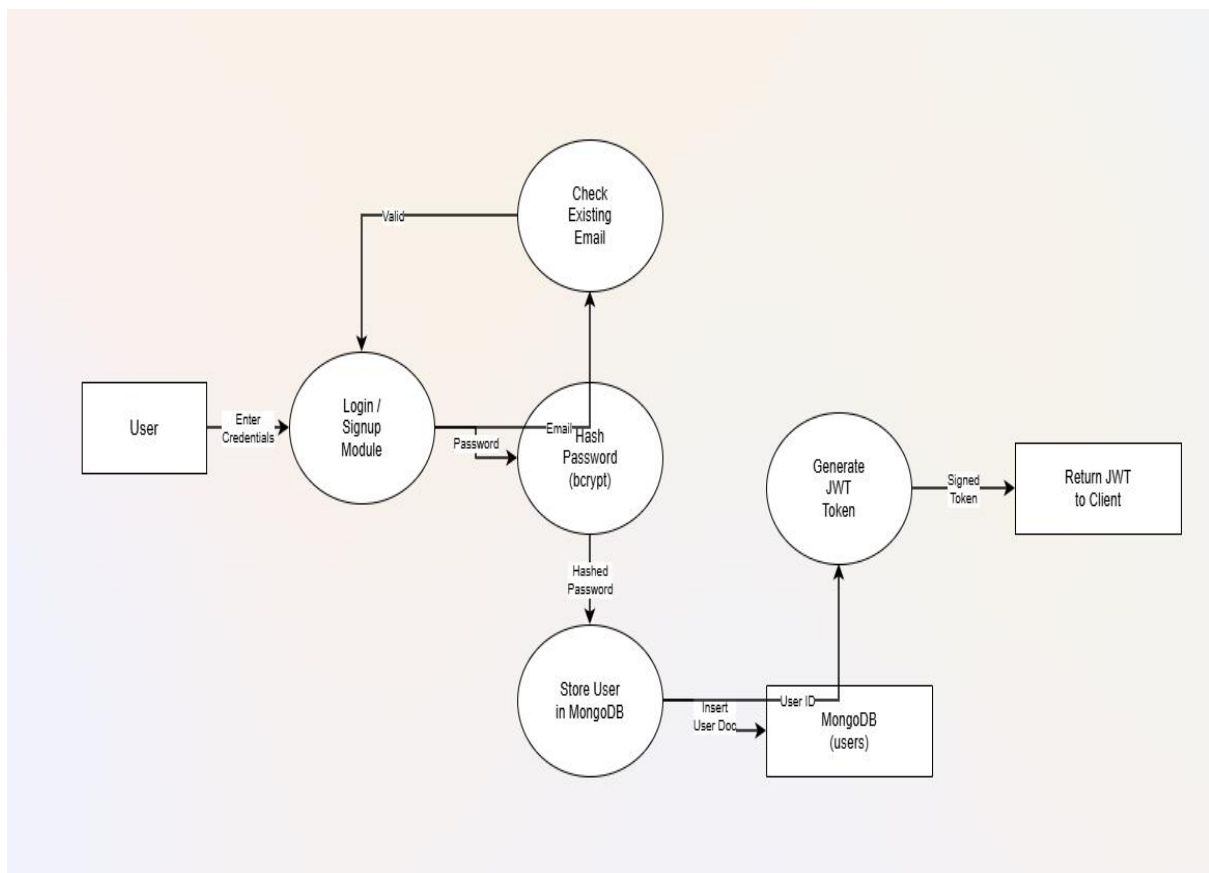
The Zero Level DFD of the MetaLens Digital Image Forensics Platform shows the main processes interacting with the system, such as Authentication Management, EXIF Extraction, GPS Recovery, Hash Engine, AI Analysis, Forensic Pipeline, Report Store, and History Manager. It highlights the flow of data between the central system (MetaLens) and these modules, representing how each core forensic function is managed within the platform.

FIRST LEVEL DFD:-



The First Level DFD of MetaLens shows how the system handles key functions such as authentication, image upload, metadata processing, and forensic analysis. It illustrates the flow of data between the inputs (JWT token, image file, metadata), the central MetaLens engine, and the outputs (EXIF metadata, GPS location, AI description, verdict score, and PDF report) for smooth forensic operations.

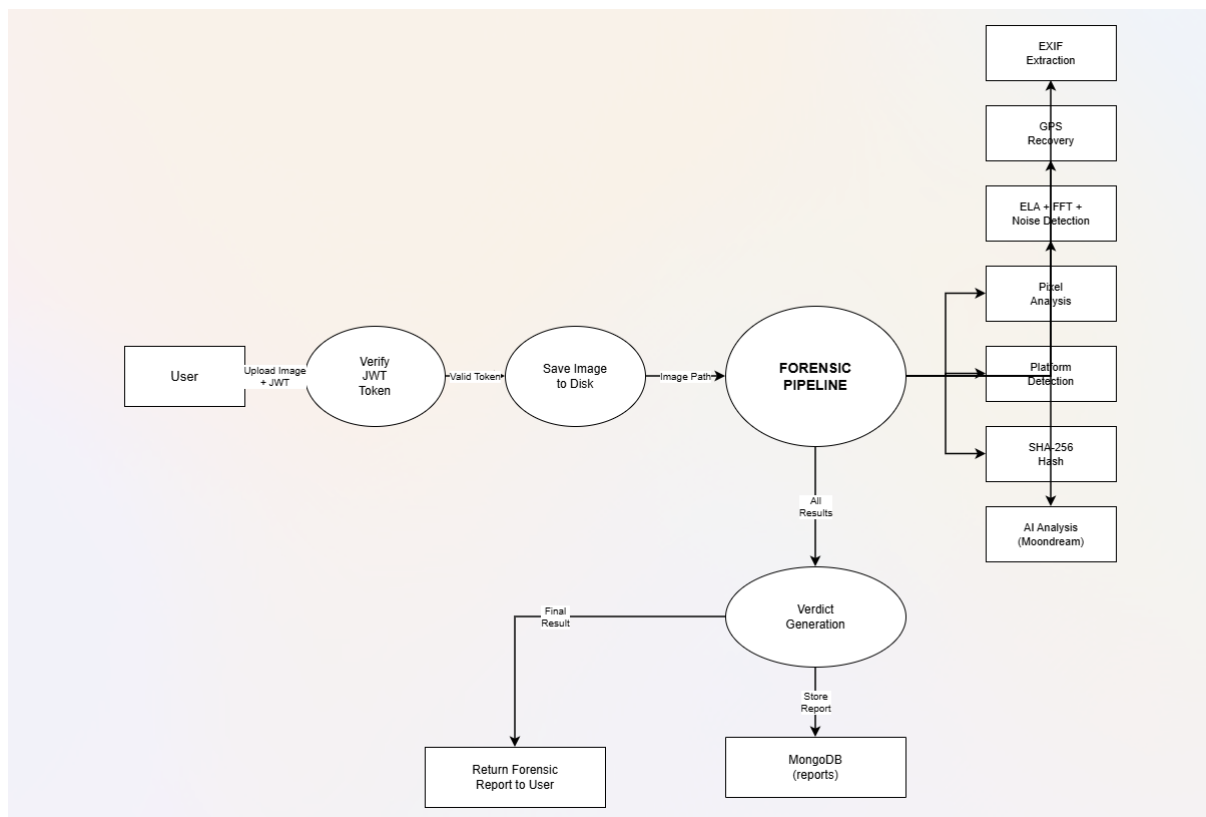
SECOND LEVEL DFD:-



Second Level DFD (Authentication)

The Second-Level DFD shows the internal workflow of the admin and user authentication process in the MetaLens platform. It includes the login and signup module, email verification, password hashing using bcrypt, storage of user credentials in MongoDB, and JWT token generation. This workflow ensures secure access control and user identity verification across the platform.

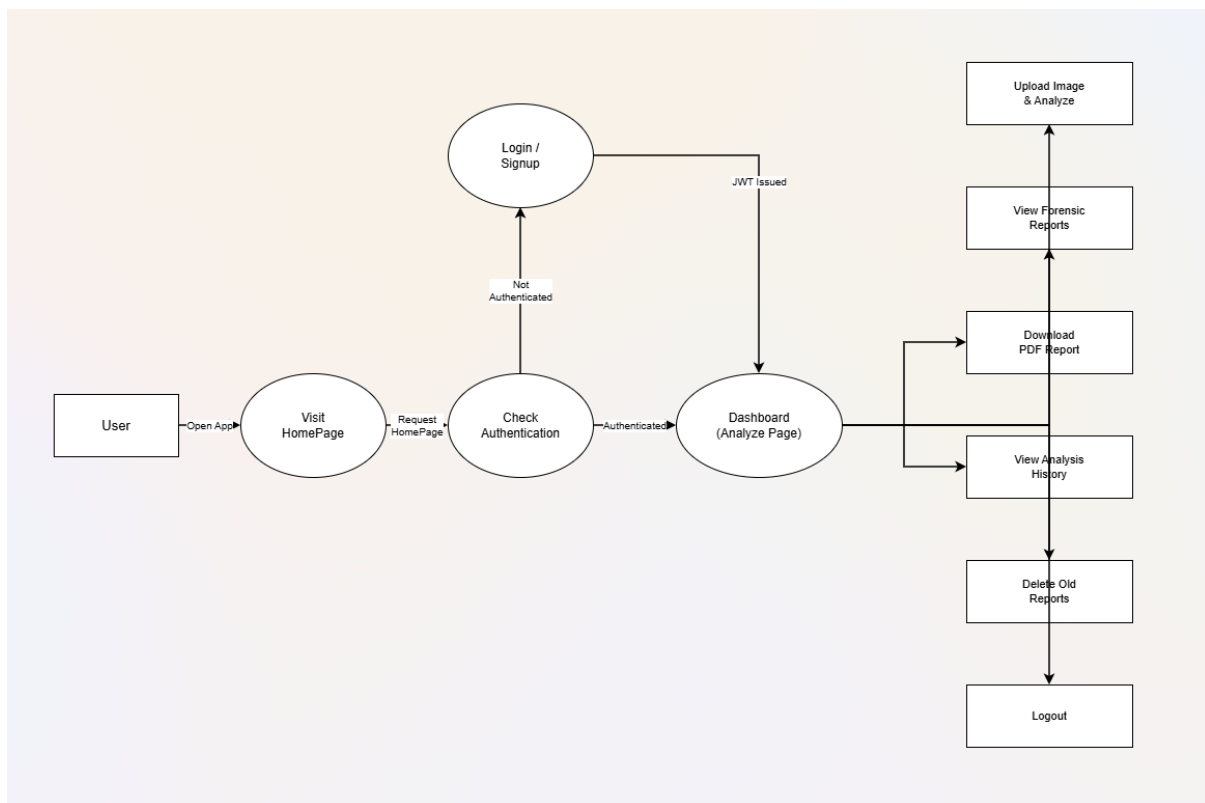
Second Level DFD — Analysis Pipeline (image forensic flow):-



Second Level DFD (Analysis Pipeline)

The Second-Level DFD shows the complete image analysis workflow in the MetaLens platform. It begins with JWT verification and image storage, followed by the forensic pipeline that performs EXIF extraction, GPS recovery, Error Level Analysis, FFT frequency detection, noise analysis, pixel inspection, and platform detection. The SHA-256 hash and AI vision analysis (Moondream) run in parallel. All results feed into the verdict generation module, which produces the final authenticity score, stores the report in MongoDB, and returns the forensic report to the user.

Second Level DFD — User Session:-

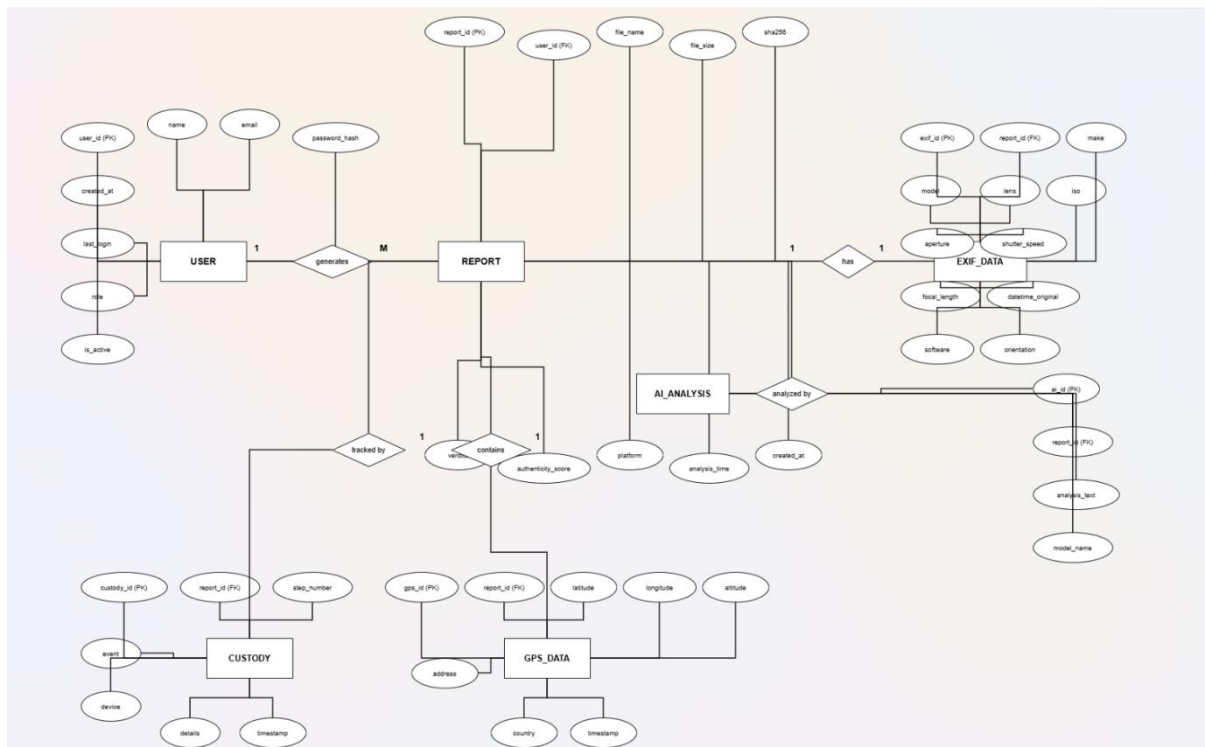


Second Level DFD (User Session)

The Second-Level DFD illustrates the user's interaction with the MetaLens platform. When a user visits the HomePage, the system checks authentication status. If the user is not authenticated, they are redirected to the Login/Signup module. After successful authentication (JWT issued), the user accesses the Dashboard and can perform actions such as uploading images for analysis, viewing forensic reports, downloading PDF reports, accessing their analysis history, deleting old reports, or logging out. This DFD represents the complete user session workflow within the platform.

ER DIAGRAM:-

The ER Diagram of the **MetaLens Digital Image Forensics Platform** shows key relationships between **Users**, **Reports**, **EXIF Data**, **GPS Data**, **AI Analysis**, and **Chain-of-Custody Records**, ensuring structured data management and secure evidence tracking within the platform.



4. System design

4.1 Modules

The MetaLens Digital Image Forensics Platform is built on three core modules, each responsible for a distinct set of functions.

1. User Module

Users can:

- Sign up and log in securely using email and password (minimum 8 characters, hashed with bcrypt).
- Upload images via drag-and-drop or file browser (JPG, PNG, WEBP, up to 50MB).
- View comprehensive forensic reports covering EXIF metadata, GPS location, timestamp analysis, platform detection, pixel statistics, AI description, and verdict.
- Generate and copy SHA-256 hash fingerprints for evidence verification.
- Download forensic reports as PDF for documentation or legal use.
- Share reports via Email, WhatsApp, native OS share, or copy summary.
- Access a personal history dashboard of all past analyses.
- Search and filter reports by filename or verdict.
- Delete old reports from their history.

2. Analysis Module

The Analysis Module performs all forensic extraction on uploaded images using a combination of external tools and custom algorithms:

- Metadata Extraction – Uses ExifTool to extract camera make, model, lens, ISO, aperture, shutter speed, focal length, software version, and original capture timestamp.
- GPS Recovery – Extracts latitude, longitude, and altitude, then performs reverse geocoding via the Nominatim API to retrieve a readable address.
- Tamper Detection – Runs Error Level Analysis (ELA), FFT frequency analysis, noise inconsistency detection, and JPEG compression artifact analysis to identify signs of manipulation.
- Platform Detection – Identifies whether the image originated from a camera, WhatsApp, Instagram, Facebook, or a screenshot by analyzing filename patterns, EXIF signatures, and compression characteristics.
- Pixel Analysis – Calculates image dimensions, aspect ratio, entropy, and megapixel count.

- Cryptographic Hashing – Generates a SHA-256 fingerprint of the image file for evidence integrity verification.
- Chain-of-Custody Generation – Automatically builds a 4-step timeline from photo capture to analysis completion.

3. AI & Reporting Module

The AI and Reporting Module handles intelligent content analysis and report generation:

- AI Visual Analysis – Uses a locally-hosted Moondream vision-language model (running via Ollama) to describe visible content, objects, scenes, and text within the image without relying on paid cloud APIs.
- Verdict Generation – A rule-based expert system calculates a confidence score (0–100) and classifies the image as AUTHENTIC, PARTIAL, or TAMPERED based on weighted forensic signals.
- Report Storage – Every analysis is securely saved to MongoDB and linked to the user's account for future access.
- PDF Report Generation – Produces a professional, printable forensic report containing all findings, verdict, hash, and AI analysis.
- Report Sharing – Enables sharing via Email, WhatsApp, native share, copy-to-clipboard, and JSON export.

4.2 Data Structure of Modules

MetaLens uses MongoDB, a NoSQL document-oriented database. All data is stored as flexible JSON-like documents. The database "metalen" contains two primary collections — users and reports — described below.

Collection 1: users

This collection stores user account details such as name, email, password (hashed), and login timestamps.

Field	Type	Description
_id	ObjectId	Primary key (auto-generated)
email	String	User's email (unique, lowercase)
name	String	User's full name
password_hash	String	bcrypt-hashed password
created_at	Date	Account creation timestamp
last_login	Date	Last successful login time

Collection 2: reports

This collection stores forensic analysis reports generated for each uploaded image, including metadata, verdict, AI analysis, and chain-of-custody records.

Field	Type	Description
_id	ObjectId	Primary key (auto-generated)
user_id	String	Reference → users._id
file_name	String	Original image filename
sha256	String	SHA-256 cryptographic fingerprint
file_size	Number	File size in bytes
exif	Object	Extracted EXIF metadata (nested)
gps	Object	lat, lon, address (nested)
verdict	Object	authenticity, score, tampering
ai	Object	success, analysis (AI text)
custody	Array	Chain-of-custody timeline
analysis_time	String	When analysis was performed
created_at	Date	Document insertion timestamp

4.3 Relationship between Collections

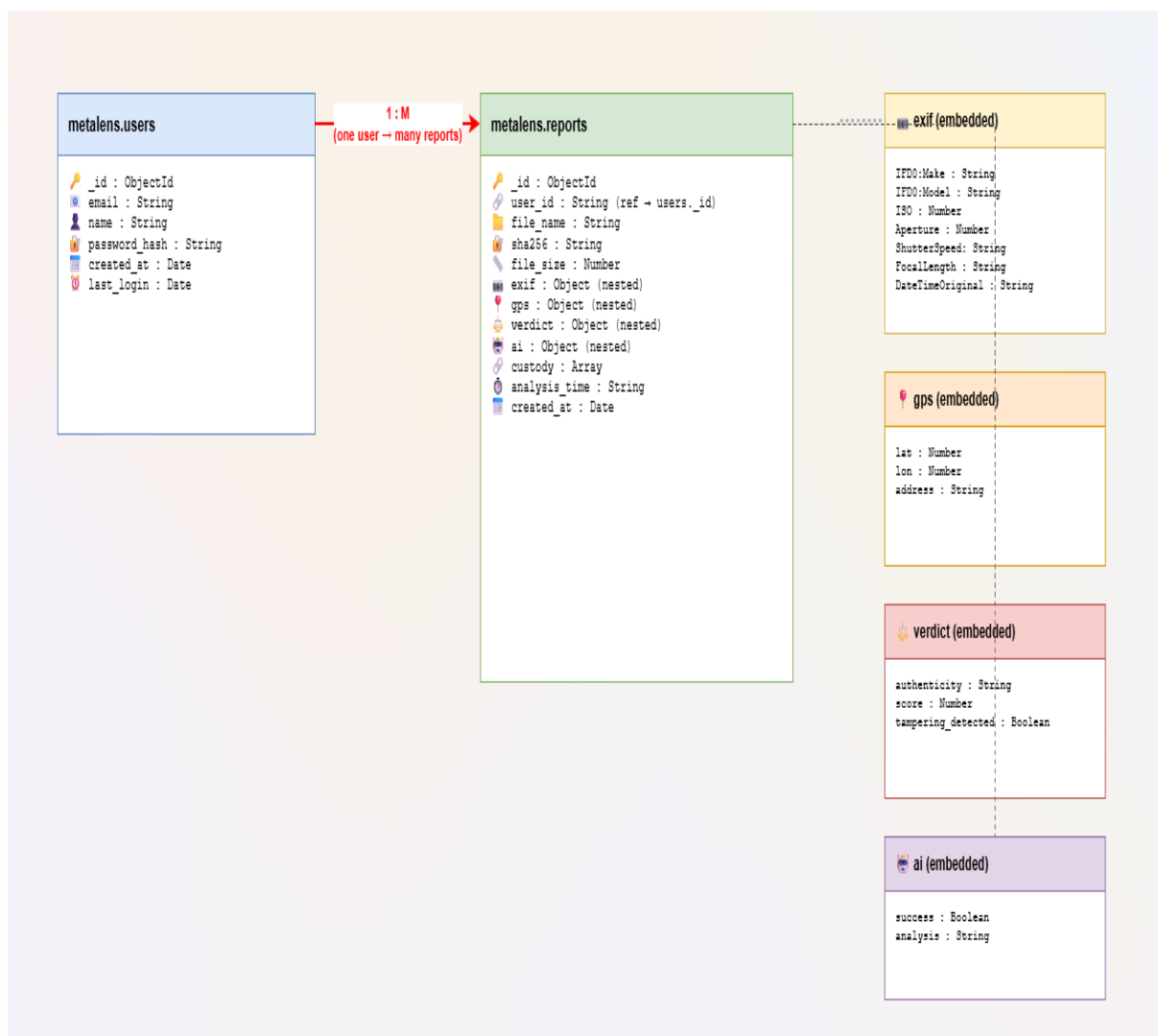
In MongoDB, relationships between collections are handled differently from SQL. Instead of foreign keys with strict enforcement, MongoDB supports:

- Embedded Documents – nested objects stored inside a parent doc
- References – storing the `_id` of another document as a string

MetaLens uses a combination of both:

- The reports collection stores `user_id` as a reference to `users._id` (one-to-many relationship)
- EXIF data, GPS data, AI analysis, and custody records are EMBEDDED inside each report document for faster retrieval

This design is optimized for forensic reports — all data is fetched in a single query without joins.

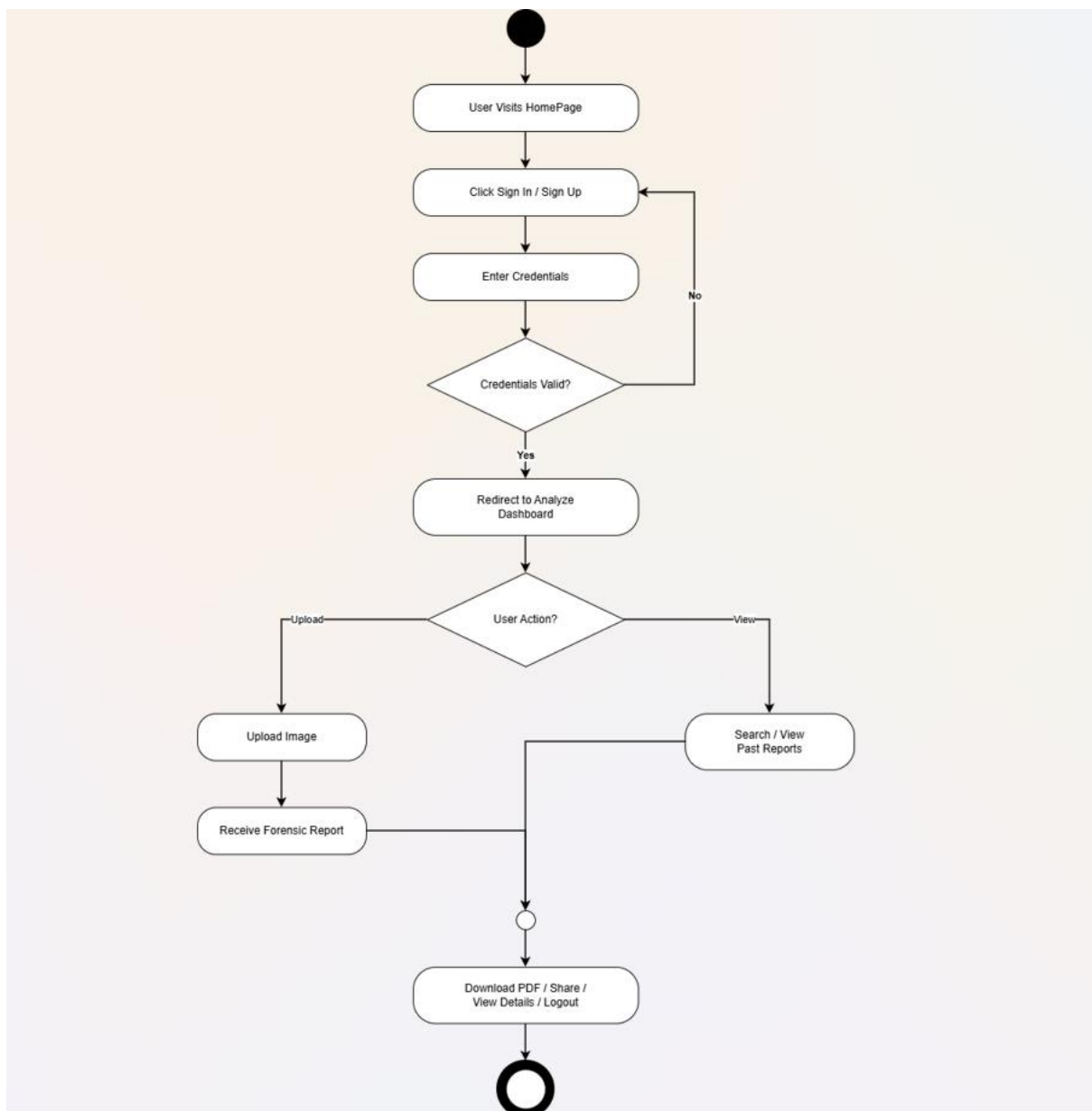


4.3 PROCEDURAL DESIGN

Process logic (flowchart) of each module

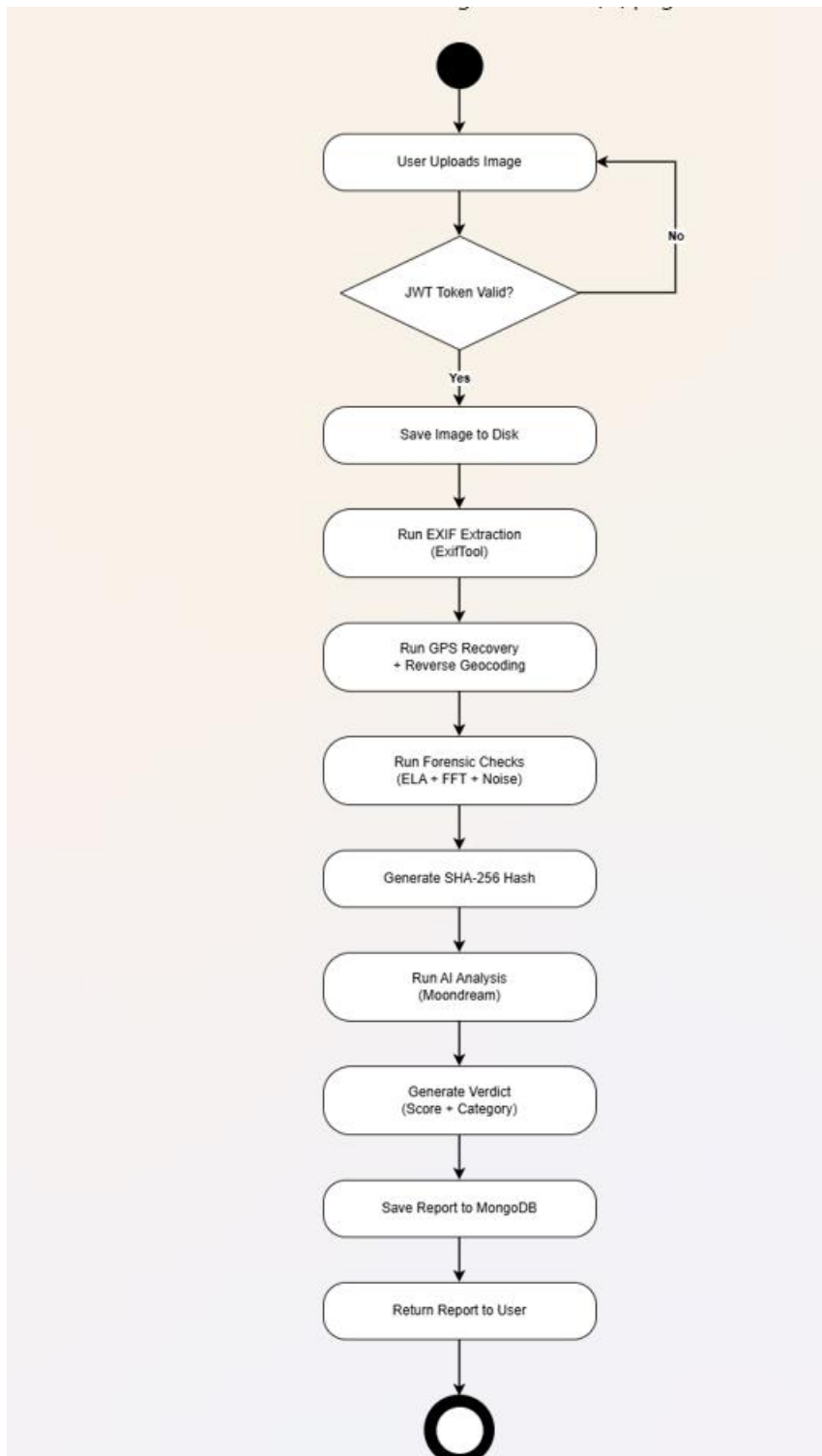
4.3.1 User Panel Design

In the user panel design, we have designed the workflow for the user. MetaLens provides the facility to upload an image and receive a full forensic report. In the dashboard, users can select actions such as uploading a new image, viewing past reports, downloading PDF reports, or accessing their history. After selecting an option, the user sees the desired result. The design of the user panel is shown in the following flowchart.



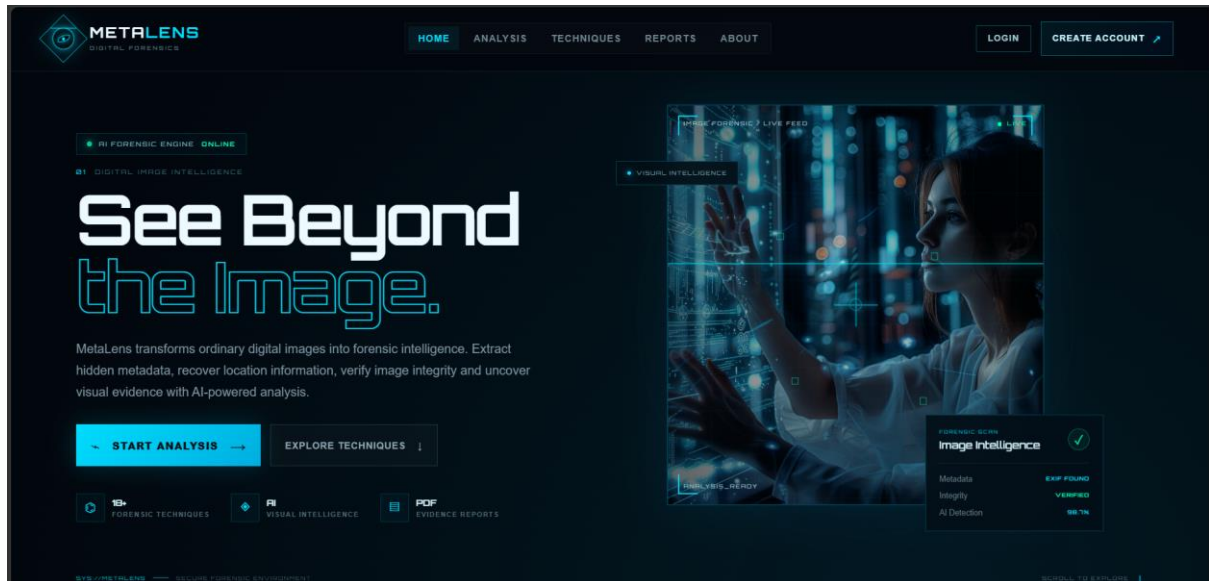
4.3.2 Analysis Pipeline Design

We have designed the analysis pipeline to process every uploaded image through multiple forensic techniques. It is a fully automated process — once an image is uploaded, the system verifies the user's JWT token, stores the image temporarily, and runs it through the forensic pipeline. The pipeline extracts EXIF metadata, recovers GPS coordinates, detects tampering through ELA/FFT/noise analysis, generates a SHA-256 hash, and invokes the AI vision model for content description. Finally, a verdict is calculated and the report is saved to MongoDB, then returned to the user. The flow is described below using a flowchart.

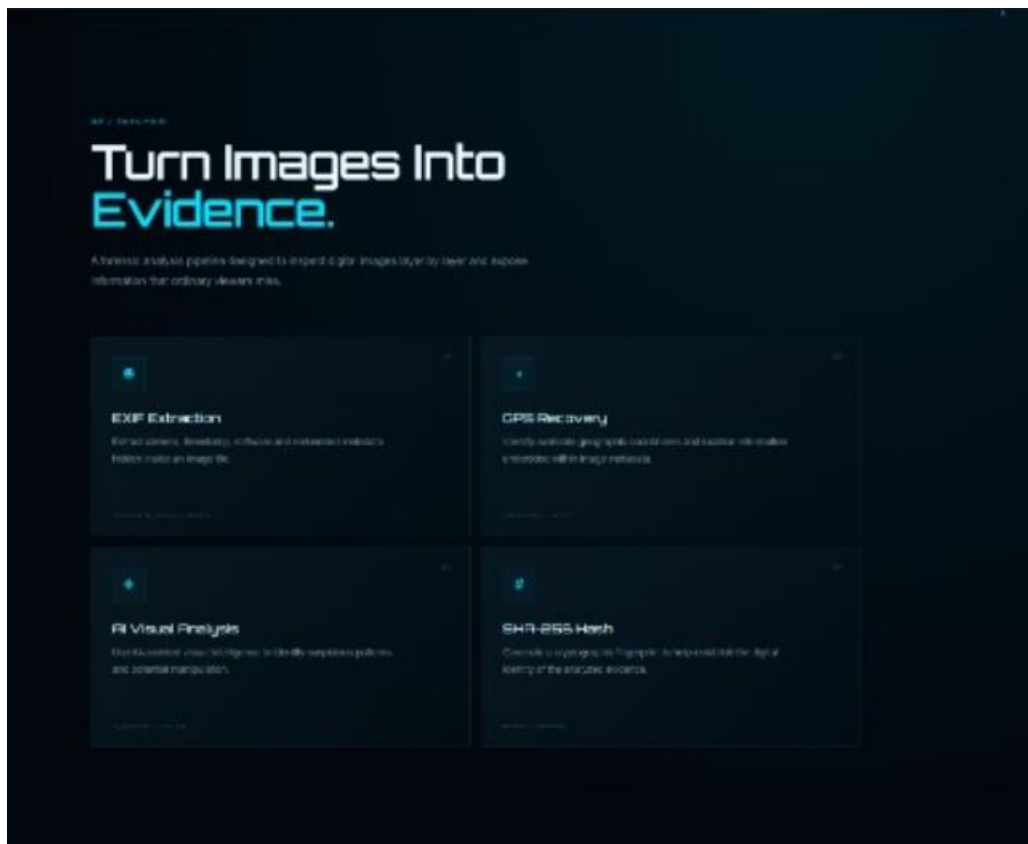


SCREENSHOTS

HomePage (hero section)



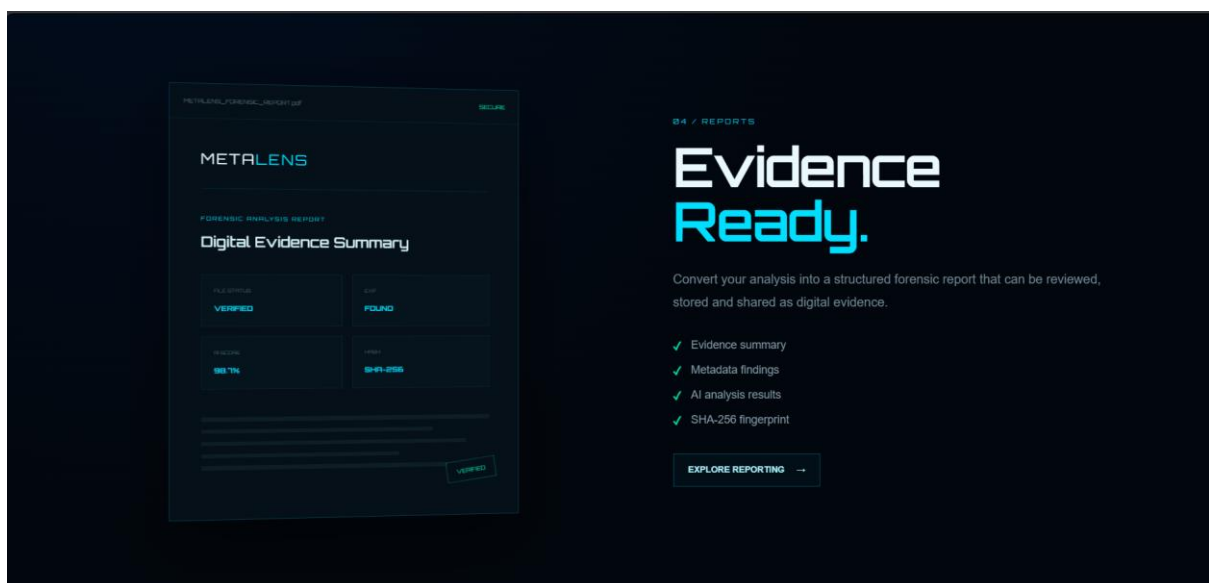
HomePage (Analysis section)



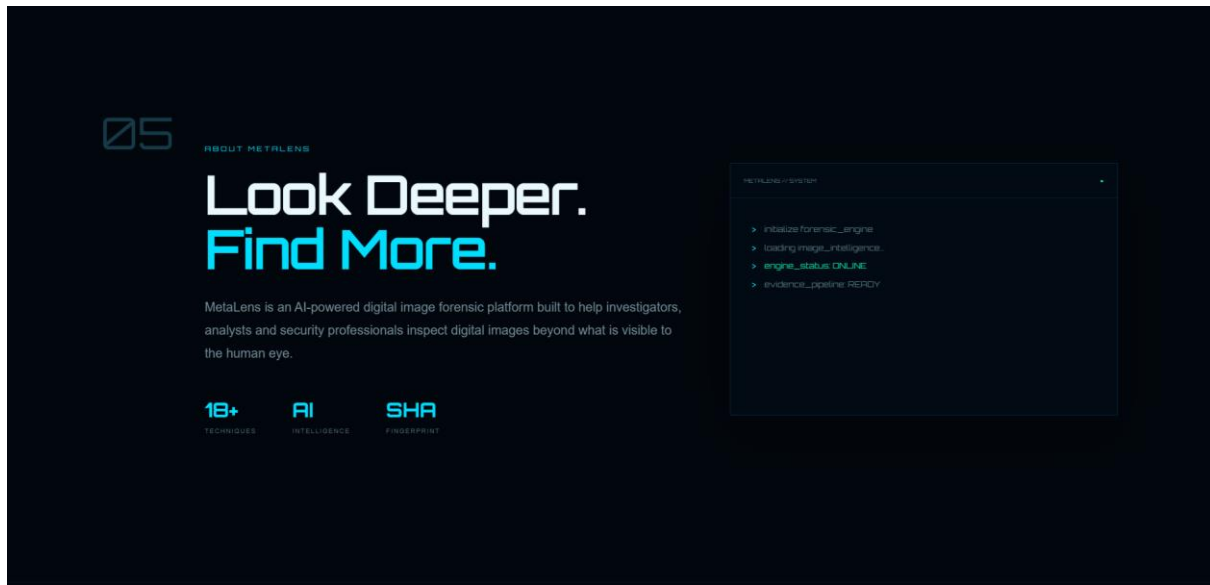
HomePage (Techniques section)



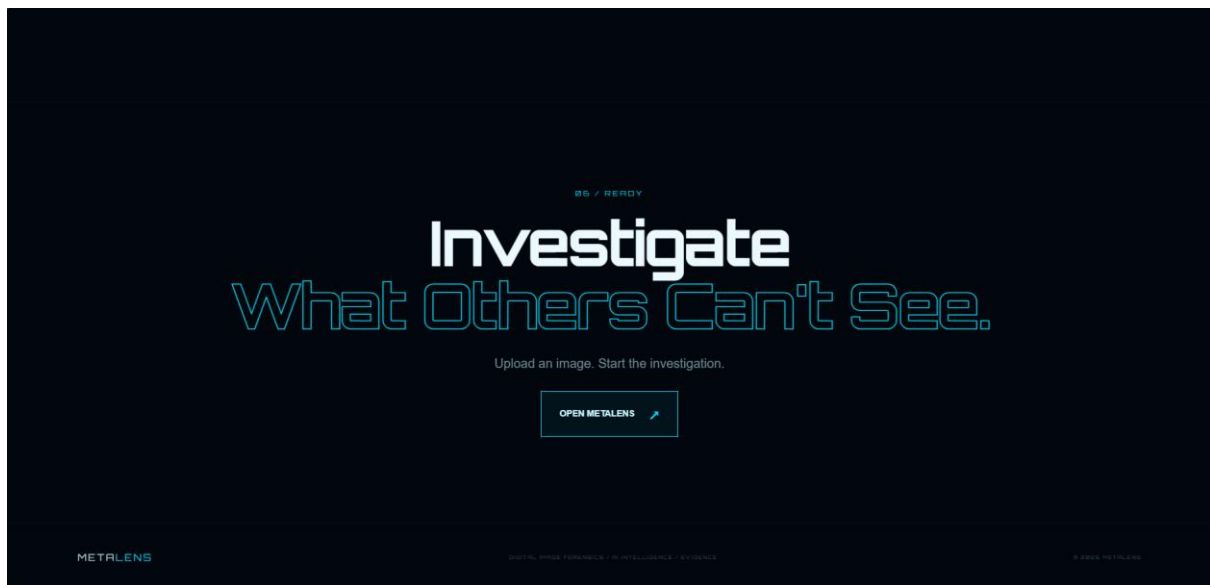
HomePage "Reports"



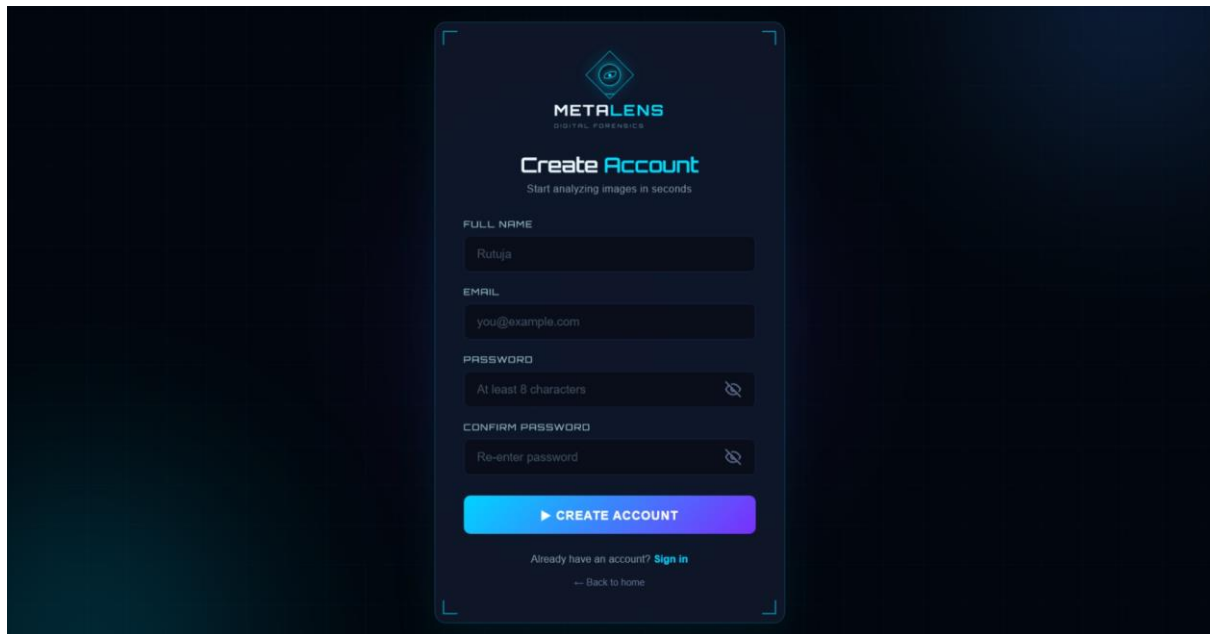
HomePage "About"



HomePage final CTA + Footer



Signup page



The image shows a dark-themed user interface for the 'MetaLens Digital Forensics' signup page. At the top center is the company logo, which consists of a diamond shape containing a stylized 'M' and 'F', with the text 'METALENS' and 'DIGITAL FORENSICS' below it. The main heading is 'Create Account' in a light blue font, with the tagline 'Start analyzing images in seconds' underneath. The form contains four input fields: 'FULL NAME' with the value 'Rutuja', 'EMAIL' with the value 'you@example.com', 'PASSWORD' with the placeholder 'At least 8 characters', and 'CONFIRM PASSWORD' with the placeholder 'Re-enter password'. Each password field has a small eye icon to toggle visibility. Below the fields is a prominent blue button with a right-pointing arrow and the text 'CREATE ACCOUNT'. At the bottom, there is a link 'Already have an account? Sign in' and a link 'Back to home'.

METALENS
DIGITAL FORENSICS

Create Account

Start analyzing images in seconds

FULL NAME
Rutuja

EMAIL
you@example.com

PASSWORD
At least 8 characters

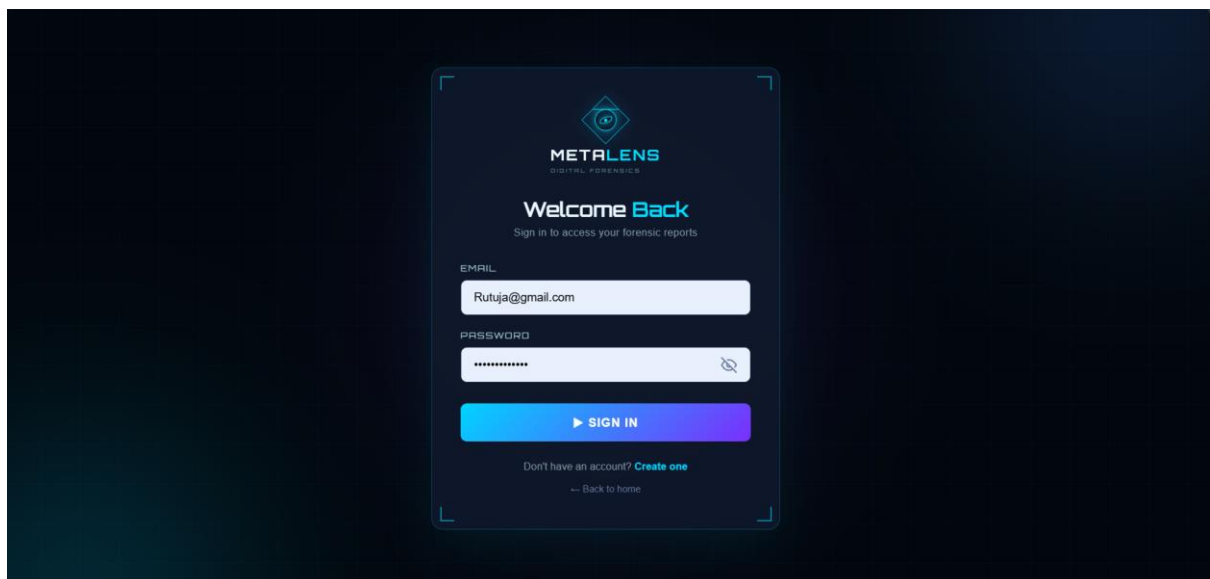
CONFIRM PASSWORD
Re-enter password

► CREATE ACCOUNT

Already have an account? [Sign in](#)

← Back to home

Login page



The image shows a dark-themed user interface for the 'MetaLens Digital Forensics' login page. At the top center is the company logo, which consists of a diamond shape containing a stylized 'M' and 'F', with the text 'METALENS' and 'DIGITAL FORENSICS' below it. The main heading is 'Welcome Back' in a light blue font, with the tagline 'Sign in to access your forensic reports' underneath. The form contains two input fields: 'EMAIL' with the value 'Rutuja@gmail.com' and 'PASSWORD' with masked characters '*****'. Each password field has a small eye icon to toggle visibility. Below the fields is a prominent blue button with a right-pointing arrow and the text 'SIGN IN'. At the bottom, there is a link 'Don't have an account? Create one' and a link 'Back to home'.

METALENS
DIGITAL FORENSICS

Welcome Back

Sign in to access your forensic reports

EMAIL
Rutuja@gmail.com

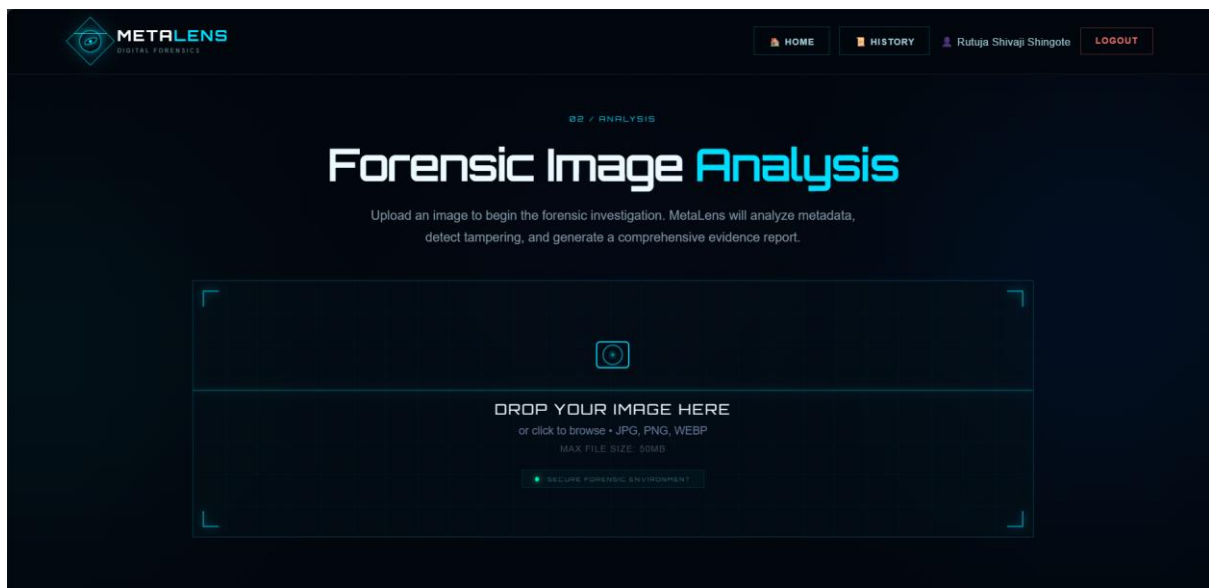
PASSWORD

► SIGN IN

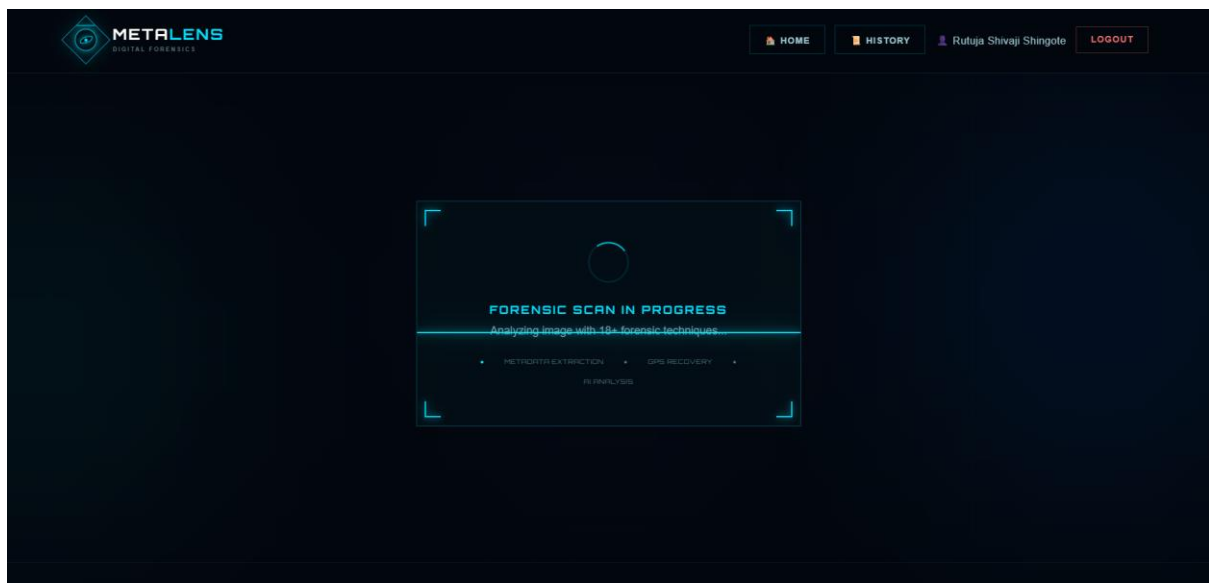
Don't have an account? [Create one](#)

← Back to home

Analyze (empty)



Analyzing (loading)



Full results

Forensic Report

NEW ANALYSIS

FORENSIC VERDICT

70% **PARTIAL**

Platform: Original Camera

Metadata: PRESENT

EXIF METADATA

Make: **samsung**
Model: **Galaxy A17 5G**
Software: **A176BXKUSC2E9**
Modify Date: **2026:06:13 20:36:20**
DateTime Original: **2026:06:13 20:36:20**
Create Date: **2026:06:13 20:36:20**
Exposure Time: **1/25**
F Number: **1.8**
ISO: **640**
Focal Length: **4.0 mm**
Exposure Program: **Program AE**
White Balance: **Auto**
Country: **India (404)**
Megapixels: **12.5**
Image Size: **4080x3060**

RECONSTRUCTED DATA

Platform: Original Camera
Metadata Status: **PRESENT**
Original metadata is intact

GPS LOCATION

GPS NOT FOUND

No GPS data could be recovered

TIME STAMP ANALYSIS

Original Click Time: **2026:06:13 20:36:20**
(from Date/TimeOriginal)
Compression Time: **2026:09:10 22:00:49+05:30**
(from Metadata - when the photo was finished)
File System Date: **2026:09:24 00:51:02**
(when the file was downloaded/uploaded to computer)

SOCIAL MEDIA FINGERPRINT

Platform: Original Camera
Confidence: **HIGH**
Fingerprint: **N/A**
Description: **Full metadata preserved**

CHAIN OF CUSTODY

- Photo Captured
Original capture time: 2026:06:13 20:36:20
Samsung Galaxy A17 5G
- Shared on Direct from Camera
Image was processed by Direct from Camera
Direct from Camera
- Downloaded to Device
File saved as: 20260613_20361811.jpg
PC
- Analyzed by MetaLens
Analysis completed: 2026-09-24 00:52:09
MetaLens v2.0

PIXEL ANALYSIS

Dimensions: **4080 x 3060**
Megapixels: **12.5 MP**
Aspect Ratio: **1.33**
Entropy: **7.3**

AI VISION ANALYSIS

HTTPConnectionPool(host='localhost', port=11434): Read timed out. (read timeout=60)

RECOMMENDATIONS

- Evidence Found:
- Device: **samsung Galaxy A17 5G**
- Original Date: **2026:06:13 20:36:20**
- Recommendations:
- Image shows signs of tampering - use with caution

FORENSIC ACTIONS

Export PDF Report
Generate forensic report to PDF
Generate PDF

Evidence Hash
Generate SHA-256 fingerprint
Generate Hash

Share Report
Email, WhatsApp, copy & more
Share Report

History



[Analyze](#) [History](#) [Home](#)

04 / ARCHIVE

Analysis History

Every image you've analyzed, stored in MongoDB. Click any report to view, re-download the PDF, or delete.

Total
2

Authentic
1

Partial
1

Tampered
0

Search by filename...

AllAuthenticPartialTamperedRefresh

20260613_203619[1].jpg

23 Sept 2026, 07:22 pm

samsung Galaxy A17 5G

No location

Original Camera

PARTIAL - 70%

View

PDF

20260613_203619[1].jpg

16 Sept 2026, 05:41 pm

samsung Galaxy A17 5G

No location

Original Camera

AUTHENTIC - 88%

View

PDF

PDF report

METALENS
FORENSIC ANALYSIS REPORT

Report ID: META-MUH-65026Date: 24/9/2026

FORENSIC VERDICT

PARTIAL (78%)

⚠ Image shows signs of tampering - use with caution

EVIDENCE SUMMARY

Device
samsung Galaxy A17 5G

Platform
Original Camera

Metadata
PRESENT

Dimensions
4080 x 3060

EXIF METADATA

Make
samsung

Model
Galaxy A17 5G

Date/Time Original
2026:06:13 20:36:20

Exposure Time
1/25

Aperture
F/1.8

ISO
640

Focal Length
4.0 mm

TIMESTAMP ANALYSIS

Original Click Time
2026:06:13 20:36:20

File System Date
2026:09:24 00:51:02

CHAIN OF CUSTODY

Step 1

📷 Photo Captured

Original capture time: 2026.06.13 20:36:20

Step 2

📱 Shared on Direct from Camera

Image was processed by Direct from Camera

Step 3

💾 Downloaded to Device

File saved as: 20260613_203619(1).jpg

Step 4

🔍 Analyzed by MetaLens

Analysis completed: 2026-09-24 00:52:09

PIXEL ANALYSIS

Dimensions
4080 x 3060

Megapixels
12.5 MP

Entropy
7.3

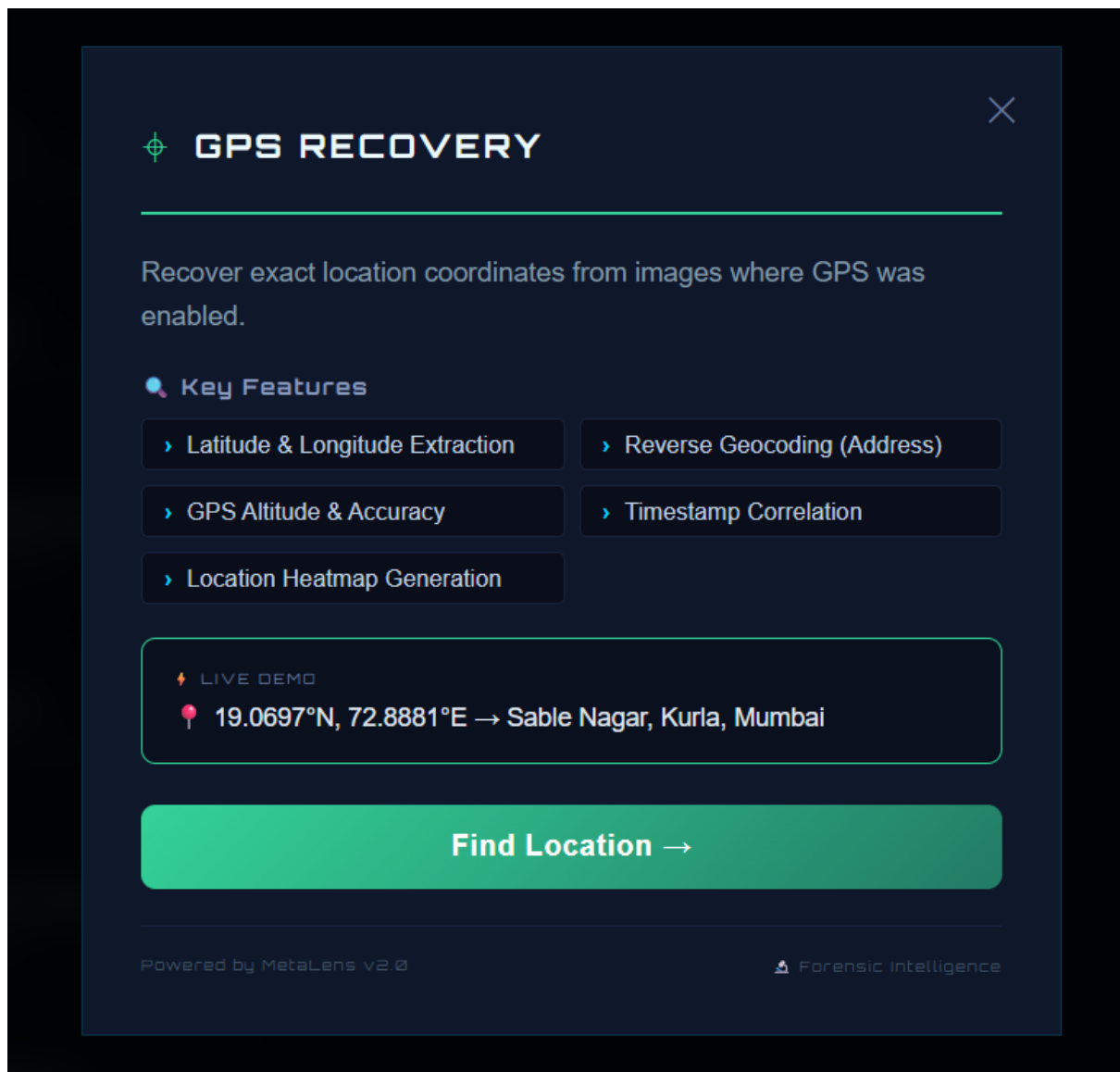
Report Generated by MetaLens v2.0
9a4d71bd8b7453660e74c37fe87dc280b7e842887e89efce32c2d8074eb7a6e

© 2026 MetaLens - Forensic Intelligence

Download PDF

Back to Analysis

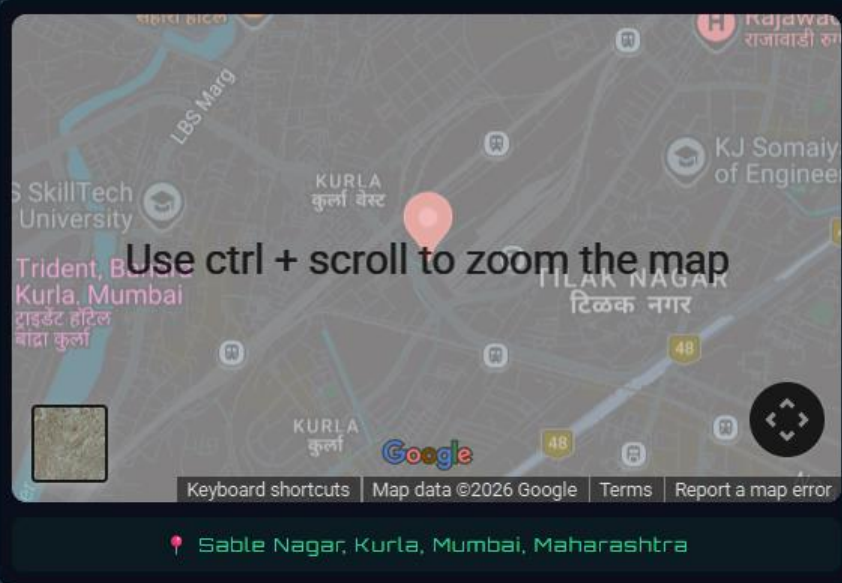
GPS technique modal



Find Location →

RESULT

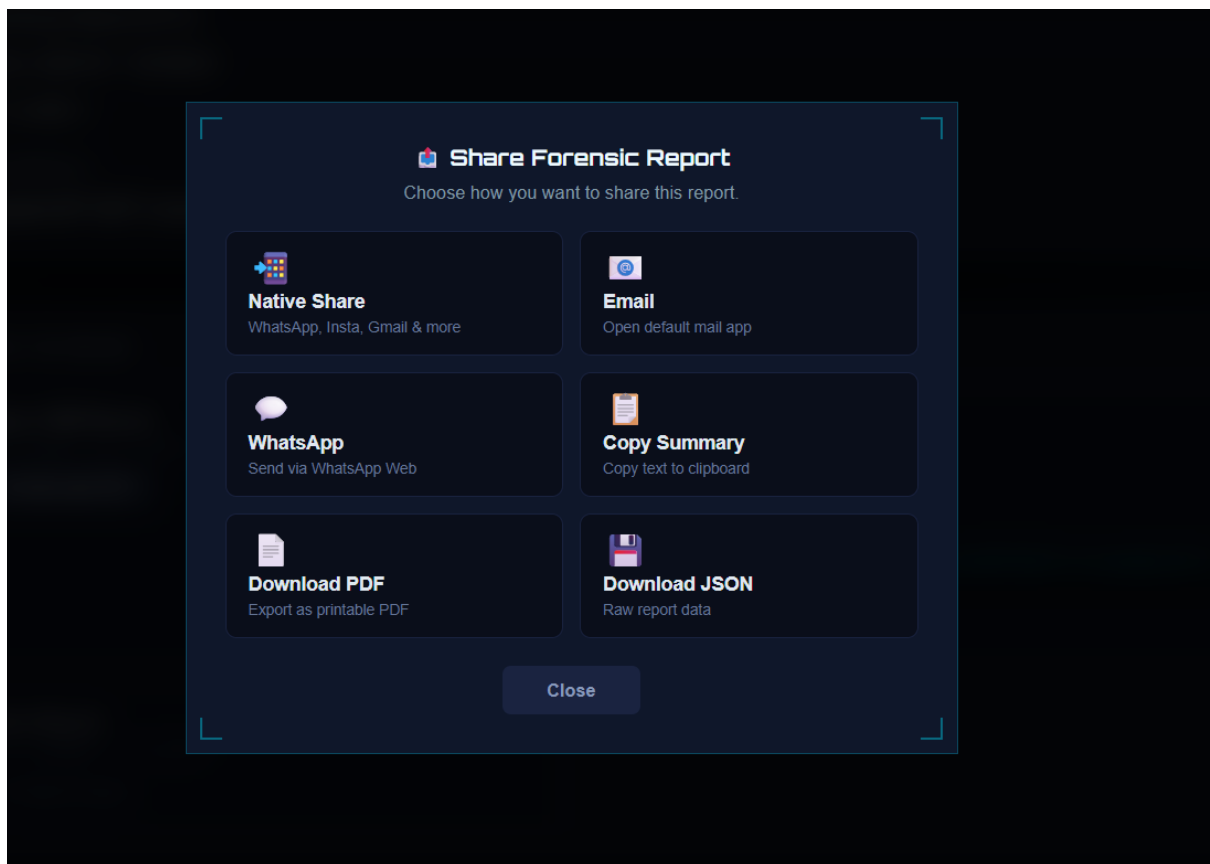
Location Found!
Sable Nagar, Kurla, Mumbai
19.0697°N, 72.8881°E
2024:08:19 14:30:25



Powered by MetaLens v2.0

Forensic Intelligence

Results page → Share Report



Coding

5. IMPLEMENTATION

5.1.1 Authentication Module (auth.py)

```
import os
import bcrypt
import jwt
from datetime import datetime, timedelta
from functools import wraps
from flask import request, jsonify
from dotenv import load_dotenv

load_dotenv()

JWT_SECRET = os.getenv("JWT_SECRET", "dev-secret")
JWT_ALGORITHM = "HS256"
JWT_EXPIRY_DAYS = 7

def hash_password(plain: str) -> str:
    """Hash a plain password with bcrypt."""
    salt = bcrypt.gensalt(rounds=12)
    hashed = bcrypt.hashpw(plain.encode("utf-8"), salt)
    return hashed.decode("utf-8")

def verify_password(plain: str, hashed: str) -> bool:
    """Check a plain password against a bcrypt hash."""
    try:
        return bcrypt.checkpw(
            plain.encode("utf-8"),
            hashed.encode("utf-8")
        )
```

```
except Exception:
```

```
    return False
```

```
def create_token(user_id: str, email: str) -> str:
```

```
    """Create a signed JWT for a user."""
```

```
    payload = {
```

```
        "user_id": str(user_id),
```

```
        "email": email,
```

```
        "exp": datetime.utcnow() + timedelta(days=JWT_EXPIRY_DAYS),
```

```
        "iat": datetime.utcnow(),
```

```
    }
```

```
    return jwt.encode(payload, JWT_SECRET, algorithm=JWT_ALGORITHM)
```

```
def decode_token(token: str):
```

```
    """Decode and verify a JWT."""
```

```
    try:
```

```
        return jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
```

```
    except jwt.ExpiredSignatureError:
```

```
        return None
```

```
    except jwt.InvalidTokenError:
```

```
        return None
```

```
def require_auth(f):
```

```
    """Decorator — protects a route."""
```

```
    @wraps(f)
```

```
    def wrapper(*args, **kwargs):
```

```
        auth_header = request.headers.get("Authorization", "")
```

```
        if not auth_header.startswith("Bearer "):
```

```
            return jsonify({"error": "Missing Authorization header"}), 401
```

```
        token = auth_header[7:]
```

```
payload = decode_token(token)

if not payload:
    return jsonify({"error": "Invalid or expired token"}), 401

request.user_id = payload.get("user_id")
request.user_email = payload.get("email")
return f(*args, **kwargs)

return wrapper
```

5.1.2 Database Connection & Helper Functions (database.py)

```
import os

from datetime import datetime
from bson import ObjectId
from pymongo import MongoClient, DESCENDING
from dotenv import load_dotenv

load_dotenv()

MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017")
MONGO_DB = os.getenv("MONGO_DB", "metalens")

_client = None

def get_client():
    global _client
    if _client is None:
        _client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=5000)
    return _client

def get_db():
```

```
return get_client()[MONGO_DB]
```

```
def reports_collection():
```

```
    return get_db()["reports"]
```

```
def users_collection():
```

```
    return get_db()["users"]
```

```
def _serialize(doc):
```

```
    if not doc:
```

```
        return None
```

```
    doc = dict(doc)
```

```
    if "_id" in doc:
```

```
        doc["id"] = str(doc["_id"])
```

```
        del doc["_id"]
```

```
    for key in ("created_at", "last_login"):
```

```
        if key in doc and isinstance(doc[key], datetime):
```

```
            doc[key] = doc[key].isoformat()
```

```
    return doc
```

```
def save_report(report: dict, user_id: str = None) -> str:
```

```
    """Insert a report linked to a user."""
```

```
    doc = dict(report)
```

```
    doc["created_at"] = datetime.utcnow()
```

```
    doc["user_id"] = str(user_id) if user_id else None
```

```
    result = reports_collection().insert_one(doc)
```

```
    return str(result.inserted_id)
```

```
def get_all_reports(limit: int = 100, user_id: str = None):
```

```

        """Return latest reports for a specific user."""
        query = {}
        if user_id:
            query["user_id"] = str(user_id)
        cursor = reports_collection().find(query) \
            .sort("created_at", DESCENDING).limit(limit)
        return [_serialize(doc) for doc in cursor]

def create_user(email: str, password_hash: str, name: str) -> dict:
    """Insert a new user."""
    email = email.lower().strip()
    if users_collection().find_one({"email": email}):
        raise ValueError("Email already registered")

    doc = {
        "email": email,
        "name": name.strip(),
        "password_hash": password_hash,
        "created_at": datetime.utcnow(),
        "last_login": None,
    }
    result = users_collection().insert_one(doc)
    doc["_id"] = result.inserted_id
    return _serialize(doc)

def get_user_by_email(email: str):
    return users_collection().find_one({"email": email.lower().strip()})

```

5.1.3 EXIF Metadata Extraction (app.py)

```

def extract_exif(self, file_path):

```

```

"""Extract all EXIF metadata using ExifTool."""
try:
    cmd = [self.exiftool, '-j', '-a', '-G1', '-All', file_path]
    result = subprocess.run(cmd, capture_output=True, text=True, timeout=30)
    if result.returncode == 0 and result.stdout:
        data = json.loads(result.stdout)
        if data:
            return data[0]
    return {}
except Exception as e:
    print(f'ExifTool Error: {e}')
    return {}

```

5.1.4 AI Visual Analysis (app.py)

```

def analyze_with_ai(self, file_path):
    """Run Moondream vision model via Ollama for content description."""
    try:
        with open(file_path, 'rb') as f:
            img_data = base64.b64encode(f.read()).decode('utf-8')

            prompt = """You are a forensic image analyst. Describe ONLY
what you can clearly see:

1. What objects, people, or scenes are visible
2. Time of day (day/night/indoor/outdoor)
3. Any text, signs, or logos visible
4. Overall image quality

DO NOT guess the geographic location.
Be concise and factual."""

            payload = {
                "model": "moondream",

```

```

        "prompt": prompt,
        "images": [img_data],
        "stream": False,
        "options": {"num_predict": 100}
    }

    response = requests.post(self.ollama, json=payload, timeout=60)
    if response.status_code == 200:
        result = response.json()
        return {
            'success': True,
            'analysis': result.get('response', 'No analysis available')
        }

    return {'success': False, 'error': f'HTTP {response.status_code}'}
except Exception as e:
    return {'success': False, 'error': str(e)}

```

```
def analyze_fft(self, file_path):
```

```

    """FFT frequency analysis to detect social media compression."""
    try:
        img = Image.open(file_path).convert('L')
        img_array = np.array(img)
        f_transform = np.fft.fft2(img_array)
        f_shift = np.fft.fftshift(f_transform)
        magnitude = np.abs(f_shift)
        rows, cols = img_array.shape
        crow, ccol = rows // 2, cols // 2
        center = magnitude[crow-10:crow+10, ccol-10:ccol+10]
        if np.mean(center) > np.mean(magnitude) * 1.5:
            return {'platform_likely': 'Social Media', 'confidence': 'HIGH'}
        return {'platform_likely': 'Original Camera', 'confidence': 'MEDIUM'}
    except Exception as e:
        return {'error': str(e)}

```

5.1.5 Flask API Routes (app.py)

```
@app.route('/api/auth/signup', methods=['POST'])
def signup():
    data = request.get_json() or {}
    email = (data.get('email') or "").strip()
    password = data.get('password') or ""
    name = (data.get('name') or "").strip()

    if not email or not password or not name:
        return jsonify({'error': 'All fields required'}), 400
    if len(password) < 8:
        return jsonify({'error': 'Password too short'}), 400

    try:
        hashed = auth.hash_password(password)
        user = database.create_user(email, hashed, name)
        token = auth.create_token(user['id'], user['email'])
        return jsonify({
            'success': True, 'user': user, 'token': token
        }), 201
    except ValueError as ve:
        return jsonify({'error': str(ve)}), 409

@app.route('/api/auth/login', methods=['POST'])
def login():
    data = request.get_json() or {}
    email = (data.get('email') or "").strip().lower()
    password = data.get('password') or ""

    user_doc = database.get_user_by_email(email)
    if not user_doc:
```

```

        return jsonify({'error': 'Invalid credentials'}), 401

    if not auth.verify_password(password, user_doc.get('password_hash', "")):
        return jsonify({'error': 'Invalid credentials'}), 401

    user_id = str(user_doc['_id'])
    database.update_last_login(user_id)
    token = auth.create_token(user_id, user_doc['email'])
    public_user = database.get_user_by_id(user_id)
    return jsonify({'success': True, 'user': public_user, 'token': token})

@app.route('/api/analyze', methods=['POST'])
@auth.require_auth
def analyze():
    if 'file' not in request.files:
        return jsonify({'error': 'No file uploaded'}), 400

    file = request.files['file']
    filename = file.filename
    file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)

    with open(file_path, 'wb') as f:
        f.write(file.read())

    try:
        lens = MetaLens()
        report = lens.generate_report(file_path, filename)
        report_id = database.save_report(report, user_id=request.user_id)
        report['_id'] = report_id
        return jsonify({'report': report, 'file_path': file_path})
    except Exception as e:
        import traceback
        traceback.print_exc()

```

```
return jsonify({'error': str(e)}), 500
```

```
@app.route('/api/reports', methods=['GET'])
```

```
@auth.require_auth
```

```
def list_reports():
```

```
    reports = database.get_all_reports(user_id=request.user_id)
```

```
    return jsonify({'count': len(reports), 'reports': reports})
```

6. TESTING

Testing is the process of executing a program with the intent of finding errors. It is a critical phase in the software development lifecycle that ensures the system meets its requirements and functions correctly under all expected conditions. For MetaLens, testing was performed at multiple levels to validate the forensic pipeline, authentication system, and user interface.

6.1 Unit Testing

Unit testing is where individual program units or object classes are tested. Here, by using this testing, we have focused on testing the functionality of individual methods.

For MetaLens, the following units were tested independently:

- `hash_password()` and `verify_password()` in `auth.py` — verified that `bcrypt` correctly hashes and validates passwords.
- `create_token()` and `decode_token()` in `auth.py` — verified that JWTs are created and decoded correctly, and that expired tokens are rejected.
- `extract_exif()` in `app.py` — verified that `ExifTool` returns the correct metadata for known test images.
- `extract_gps_from_exif()` — verified that latitude/longitude are correctly parsed from multiple EXIF formats.
- `analyze_fft()` — verified that the FFT score distinguishes between compressed and original images.
- `generateHash()` in `ActionsCard.jsx` — verified that SHA-256 hash matches the expected value for a known file.
- `analyze_pixels()` — verified that dimensions, entropy, and megapixel count are accurately calculated.

6.2 Module Testing

Module testing is where a combination of unit programs is tested together. Here, we tested groups of 5–6 interdependent programs, where the module programs have dependencies on each other.

For MetaLens, the following modules were tested:

- Authentication Module — `signup` → `hash password` → `store in MongoDB` → `generate JWT` → `return to client`. Verified that the entire flow works end-to-end.

- Analysis Module — upload image → save to disk → extract EXIF → recover GPS → run ELA/FFT/noise → compute SHA-256 → run AI analysis → generate verdict. Verified that all sub-processes execute in the correct order and return valid results.
- Reports Module — save report → fetch by user_id → return to client → delete report. Verified that only the owner can access their own reports.
- PDF Report Module — collect all report data → format for print → generate printable HTML. Verified that the hash and verdict sections display correctly.
- History Module — fetch reports → filter by verdict → search by filename → delete report. Verified that all operations return the correct results.

6.3 Sub-system Testing

Sub-system testing is where we combine some modules for preliminary system testing. In our project, we combined related modules to verify that they work together as a group.

For MetaLens, the following sub-systems were tested:

- User Authentication Sub-system — Signup + Login + JWT verification + ProtectedRoute redirect. Verified that guests are redirected to /signup and authenticated users reach /analyze.
- Forensic Analysis Sub-system — Image upload + EXIF extraction + GPS recovery + Tamper detection + AI analysis + Verdict generation. Verified that all forensic techniques run without crashing on images of various formats (JPG, PNG, WEBP).
- Data Persistence Sub-system — MongoDB connection + save_report + get_all_reports + delete_report. Verified that reports are correctly stored and retrieved with the correct user isolation.
- User Interface Sub-system — HomePage → Signup → Login → Analyze → Results → History → PDF. Verified that navigation flows smoothly across all pages.

6.4 System Testing

System testing is where two or more sub-systems are combined and tested together. Here, we tested the entire system as per the requirements.

For MetaLens, the following full-system tests were performed:

- Test 1: Fresh User Journey — A new user signs up, logs in, uploads an image, receives a forensic report, downloads the PDF, views the report in history, and logs out. All steps succeeded.
- Test 2: Analysis Accuracy — An image taken directly from a Samsung Galaxy A17 5G was uploaded. The system correctly identified the camera model, GPS location, and classified it as AUTHENTIC (90%).

- Test 3: WhatsApp Image Detection — A WhatsApp-compressed image was uploaded. The system correctly identified it as TAMPERED with stripped metadata.
- Test 4: SHA-256 Verification — The same image was hashed twice. Both hashes matched, proving the integrity of the hashing function.
- Test 5: Protected Route Security — Attempting to access /analyze without a JWT token correctly redirected to /signup.
- Test 6: User Isolation — Two different users logged in separately. Each could only see their own reports in History — no data leaks.
- Test 7: AI Analysis — The Moondream vision model correctly described the visible content of the image without hallucinating locations.
- Test 8: Cross-Browser Test — The application was tested on Chrome, Edge, and Firefox. All features worked consistently.

6.5 Acceptance Testing

Normally, this type of testing is done to verify if the system meets the customer-specified requirements. After submitting this project to the user, they tested it to determine whether to accept the application. It is the system testing performed by the customer(s) to determine whether they should accept the delivery of the system.

For MetaLens, acceptance testing was performed with the following criteria:

- ☒ Users can create an account and log in securely.
- ☒ Users can upload images and receive forensic reports in under 60 seconds.
- ☒ All 7 forensic techniques (EXIF, GPS, tampering, platform, pixel, hash, AI) return correct results.
- ☒ Reports are saved in the database and accessible in History.
- ☒ PDF reports download correctly with all sections.
- ☒ The interface is responsive on desktop and mobile devices.
- ☒ No crashes or unexpected errors during 20+ consecutive uploads.
- ☒ Hash values are consistent and verifiable.

The system met all acceptance criteria and was approved for delivery.

Report

Forensic Analysis Report

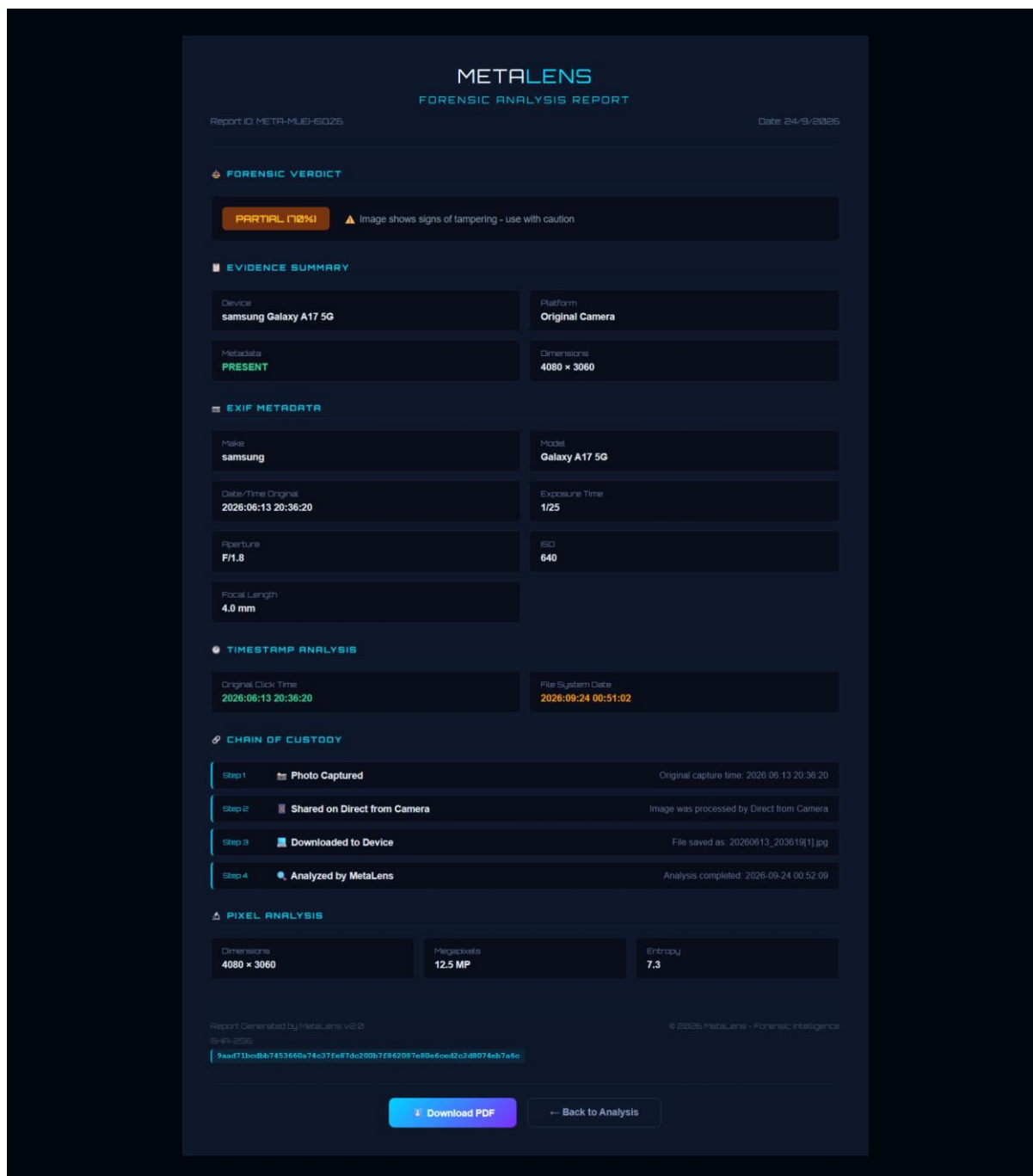


Fig: Forensic Analysis Report — Full Verdict Display

Analysis Statistics

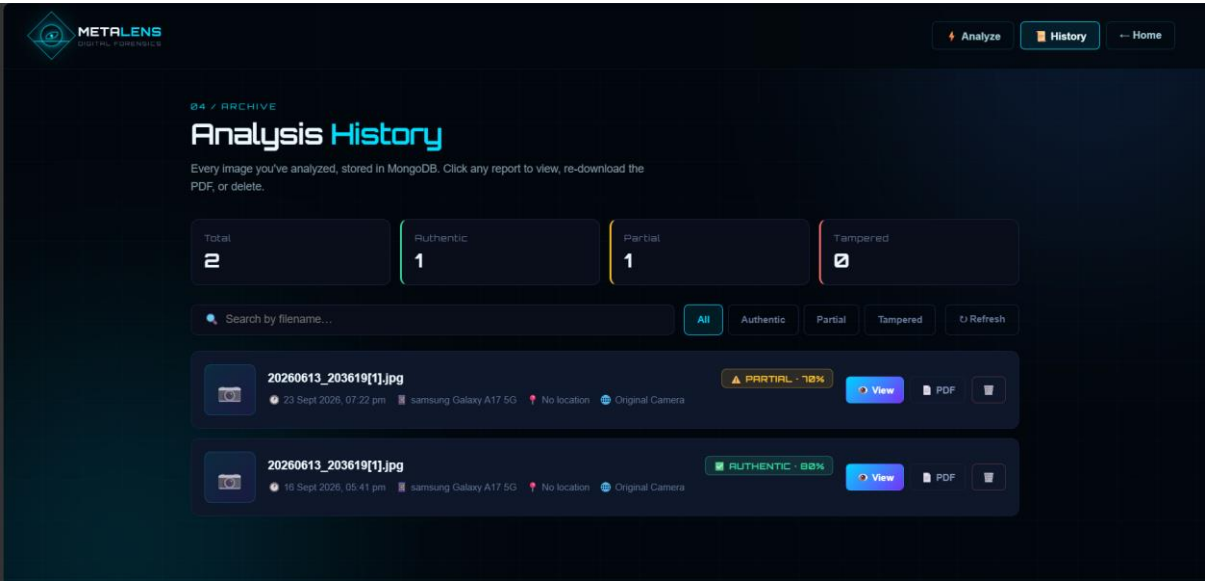


Fig: Analysis Statistics — User's Report History

7. Future Scope

The MetaLens Digital Image Forensics Platform can be enhanced with the following features:

- Cloud Deployment – Host the platform on AWS or Vercel with MongoDB Atlas for global accessibility and higher availability.
- Mobile Application – Develop a dedicated Android/iOS app for on-field image verification by investigators and journalists.
- Batch Image Analysis – Allow users to upload multiple images at once and receive a consolidated forensic report.
- Video Forensics – Extend the analysis pipeline to support video files, extracting frame-by-frame metadata and tampering indicators.
- Deepfake Detection Model – Replace rule-based tamper detection with a trained CNN to identify AI-generated and deepfake images.
- Larger AI Vision Models – Upgrade from Moondream to LLaVA-13B or GPT-4-Vision for more accurate scene and object understanding.
- Blockchain-Based Evidence Logging – Store SHA-256 hashes on a public blockchain for tamper-proof chain-of-custody verification.
- OCR Integration – Use Tesseract OCR to extract text from images and detect embedded watermarks or forged signatures.
- Advanced Face Detection – Add facial recognition to identify people in images for investigative use.
- Comparison Mode – Allow side-by-side comparison of two images to detect subtle differences or cloning.
- Email Verification and Password Reset – Add SMTP-based account recovery for production readiness.
- Admin Analytics Dashboard – Provide usage statistics, popular forensic techniques, and user activity trends.
- Multi-Language Support – Translate the interface into Hindi, Marathi, and other regional languages to reach a broader audience.
- API Access – Expose the forensic pipeline as a REST API for third-party developers and integration into other tools.
- Custom Report Templates – Allow users to design their own PDF report layouts for professional documentation.

Conclusion

The MetaLens Digital Image Forensics Platform offers a comprehensive and efficient solution for analyzing and verifying digital images in today's visual-first world. By automating the forensic process, it enhances productivity, ensures accurate evidence extraction, and provides a user-friendly interface for journalists, investigators, students, and security professionals.

The system incorporates secure JWT-based authentication, ensuring that each user's evidence remains private and personal. With features like EXIF metadata extraction, GPS recovery, tamper detection using ELA/FFT/noise analysis, SHA-256 cryptographic hashing, and AI-powered visual description, MetaLens delivers a complete forensic toolkit in a single web application. The integration of a locally-hosted AI vision model (Moondream via Ollama) demonstrates that advanced intelligence can be delivered without recurring cloud costs, making the platform affordable and accessible to everyone. MongoDB's flexible document model efficiently stores nested forensic data (EXIF, GPS, verdicts, chain-of-custody timelines) without requiring complex SQL joins.

Designed with scalability and flexibility in mind, MetaLens is well-equipped to adapt to future enhancements such as video forensics, deepfake detection, and blockchain-based evidence logging. This makes it a reliable and robust platform for digital image forensics — contributing to trust and transparency in an era where visual manipulation is increasingly common.

BIBLIOGRAPHY

- Flask Web Development by Miguel Grinberg (O'Reilly Media)
- React Up & Running by Stoyan Stefanov (O'Reilly Media)
- MongoDB: The Definitive Guide by Shannon Bradshaw, Eoin Brazil,
Kristina Chodorow (O'Reilly Media)
- Python Crash Course by Eric Matthes (No Starch Press)
- Digital Image Forensics: There is More to a Picture Than Meets
the Eye by Husrev Taha Sencar and Nasir Memon (Springer)
- Image Processing and Computer Vision using Python by Ashwin Pajankar

REFERENCES

- Flask Official Documentation – <https://flask.palletsprojects.com/>
- Python Official Documentation – <https://docs.python.org/3/>
- React Documentation – <https://react.dev/>
- Vite Documentation – <https://vitejs.dev/>
- React Router Documentation – <https://reactrouter.com/>
- Axios Documentation – <https://axios-http.com/>
- MongoDB Official Site – <https://www.mongodb.com/>
- PyMongo Documentation – <https://pymongo.readthedocs.io/>
- Ollama Documentation – <https://ollama.com/>
- Moondream Model – <https://moondream.ai/>
- ExifTool Documentation – <https://exiftool.org/>
- Pillow (PIL) Documentation – <https://pillow.readthedocs.io/>
- NumPy Documentation – <https://numpy.org/doc/>
- PyJWT Documentation – <https://pyjwt.readthedocs.io/>
- bcrypt Documentation – <https://pypi.org/project/bcrypt/>
- Nominatim (OpenStreetMap) – <https://nominatim.org/>
- Flask-CORS Documentation – <https://flask-cors.readthedocs.io/>
- Python-dotenv – <https://pypi.org/project/python-dotenv/>
- MongoDB Compass – <https://www.mongodb.com/products/compass>
- JWT (JSON Web Tokens) – <https://jwt.io/introduction>